

Graph Algorithms

Module -4

BCSE204L – Design and Analysis of Algorithms





Outline

- Network Flow
- Network Flow Problem
- Ford-Fulkerson Algorithm
- Edmonds Karp Algorithm
- Push Re-label Algorithm



Network Flow

- In [graph theory](#), a **flow network** (also known as a **transportation network**) is a [directed graph](#) where each edge has a **capacity** and each edge receives a flow.
- The amount of flow on an edge cannot exceed the capacity of the edge. Often in [operations research](#), a directed graph is called a **network**, the vertices are called **nodes** and the edges are called **arcs**.
- A flow must satisfy the restriction that the amount of flow into a node equals the amount of flow out of it, unless it is a **source**, which has only outgoing flow, or **sink**, which has only incoming flow.
- A network can be used to model traffic in a computer network, circulation with demands, fluids in pipes, currents in an electrical circuit, or anything similar in which something travels through a network of nodes.



Network Flow

- **Definition**
- A **network** is a directed graph $G = (V, E)$ with a non-negative **capacity function** c for each edge, and without multiple arcs (i.e. edges with the same source and target nodes).
- Without loss of generality, we may assume that if $(u, v) \in E$, then (v, u) is also a member of E . Additionally, if $(v, u) \notin E$ then we may add (v, u) to E and then set the $c(v, u) = 0$.
- If two nodes in G are distinguished – one as the source s and the other as the sink t – then (G, c, s, t) is called a **flow network**.



Network Flow

- Let $G = (V, E)$ be a flow network. Let s be the source of the network, and let t be the sink.
- A flow in G is a real-valued function $f: V \times V \rightarrow R$ such that the following properties hold:
 - **Capacity Constraint:** For all $u, v \in V$, we need $f(u, v) \leq c(u, v)$.
 - **Skew Symmetry:** For all $u, v \in V$, we need $f(u, v) = -f(v, u)$.
 - **Flow Conservation:** For all $u \in V - \{s, t\}$, we need

$$\sum_{(u,v) \in V} f(u, v) = \sum_{(v,z) \in V} f(v, z) = 0$$



Network Flow

- **Flows**
- Flow functions model the net flow of units between pairs of nodes, and are useful when asking questions such as *what is the maximum number of units that can be transferred from the source node s to the sink node t ?*
- The amount of flow between two nodes is used to represent the net amount of units being transferred from one node to the other.



Network Flow

- **Flows**
- The Real value function $x_j: V \rightarrow R$ represents the net flow entering a given node u (i.e. the sum of the flows entering u) and is defined by

$$\sum_{(u,v) \in V} f(u,v) = \sum_{(v,z) \in V} f(v,z) = 0$$



Network Flow

- **Flows**
- A node u is said to be
 - **active** if $x_f(u) > 0$ (i.e. the node u consumes flow),
 - **deficient** if $x_f(u) < 0$ (i.e. the node u produces flow), or
 - **conserving** if $x_f(u) = 0$.
- In flow networks, the source s is deficient, and the sink t is active.



Network Flow

- **Pseudo-flows, feasible flows, and pre-flows are all examples of flow functions.**
- A **pseudo-flow** is a function f of each edge in the network that satisfies the following two constraints for all nodes u and v :
 - **Skew symmetry constraint:** The flow on an arc from u to v is equivalent to the negation of the flow on the arc from v to u , that is: $f(u, v) = -f(v, u)$. The sign of the flow indicates the flow's direction.
 - **Capacity constraint:** An arc's flow cannot exceed its capacity, that is: $f(u, v) \leq c(u, v)$.



Network Flow

- A **pre-flow** is a pseudo-flow that, for all $v \in V - \{s\}$, satisfies the additional constraint:
 - **Non-deficient flows:** The net flow entering the node v is non-negative, except for the source, which "produces" flow. That is: $x_f(v) \geq 0$ for all $v \in V - \{s\}$.
- A **feasible flow**, or just a flow, is a pseudo-flow that, for all $v \in V - \{s, t\}$, satisfies the additional constraint:
 - **Flow conservation constraint:** The total net flow entering a node v is zero for all nodes in the network except the source s and the sink t , that is: $x_f(v) = 0$ for all $v \in V - \{s, t\}$. In other words, for all nodes in the network except the source s and the sink t , the total sum of the incoming flow of a node is equal to its outgoing flow (i.e. $\sum_{u \in V} f(u, v) = \sum_{u \in V} f(v, u) = 0$, for each vertex $v \in V - \{s, t\}$).



Concepts used in Flow Problem

Adding arcs and flows

- We do not use multiple arcs within a network because we can combine those arcs into a single arc. To combine two arcs into a single arc, we add their capacities and their flow values, and assign those to the new arc:
 - Given any two nodes u and v , having two arcs from u to v with capacities $c_1(u, v)$ and $c_2(u, v)$ respectively is equivalent to considering only a single arc from u to v with a capacity equal to $c_1(u, v) + c_2(u, v)$.
 - Given any two nodes u and v , having two arcs from u to v with pseudo-flows $f_1(u, v)$ and $f_2(u, v)$ respectively is equivalent to considering only a single arc from u to v with a pseudo-flow equal to $f_1(u, v) + f_2(u, v)$.



Concepts used in Flow Problem

Residuals

- The **residual capacity** of an arc e with respect to a pseudo-flow f is denoted c_f , and it is the difference between the arc's capacity and its flow. That is, $c_f(e) = c(e) - f(e)$.
- From this we can construct a **residual network**, denoted $G_f(V, E_f)$, with a capacity function c_f which models the amount of *available* capacity on the set of arcs in $G = (V, E)$.
- More specifically, capacity function c_f of each arc (u, v) in the residual network represents the amount of flow which can be transferred from u to v given the current state of the flow within the network.
- This concept is used in [Ford–Fulkerson algorithm](#) which computes the [maximum flow](#) in a flow network.



Concepts used in Flow Problem

Augmenting paths

- An **augmenting path** is a path $(u_1, u_2, \dots, u_k)$ in the residual network, where $u_1 = s$, $u_k = t$, and for all u_i, u_{i+1} ($c_f(u_i, u_{i+1}) > 0$) ($1 \leq i < k$). More simply, an augmenting path is an available flow path from the source to the sink. A network is at maximum flow if and only if there is no augmenting path in the residual network G_f .
- The **bottleneck** is the minimum residual capacity of all the edges in a given augmenting path. The flow network is at maximum flow if and only if it has a bottleneck with a value greater than zero.
- The term "augmenting the flow" for an augmenting path means updating the flow f of each arc in this augmenting path to equal the capacity c of the bottleneck. Augmenting the flow corresponds to pushing additional flow along the augmenting path until there is no remaining available residual capacity in the bottleneck.



Concepts used in Flow Problem

Multiple sources and/or sinks

- Sometimes, when modeling a network with more than one source, a **supersource** is introduced to the graph. This consists of a vertex connected to each of the sources with edges of infinite capacity, so as to act as a global source. A similar construct for sinks is called a **supersink**.



Network Flow Problem

- A type of network optimization problem
- It arise in many different domains like
 - **Networks:** routing as many packets as possible on a given network
 - **Transportation:** sending as many trucks as possible, where roads have limits on the number of trucks per unit time.
 - **Bridges:** destroying some bridges to disconnect s from t while minimizing the cost of destroying the bridges.

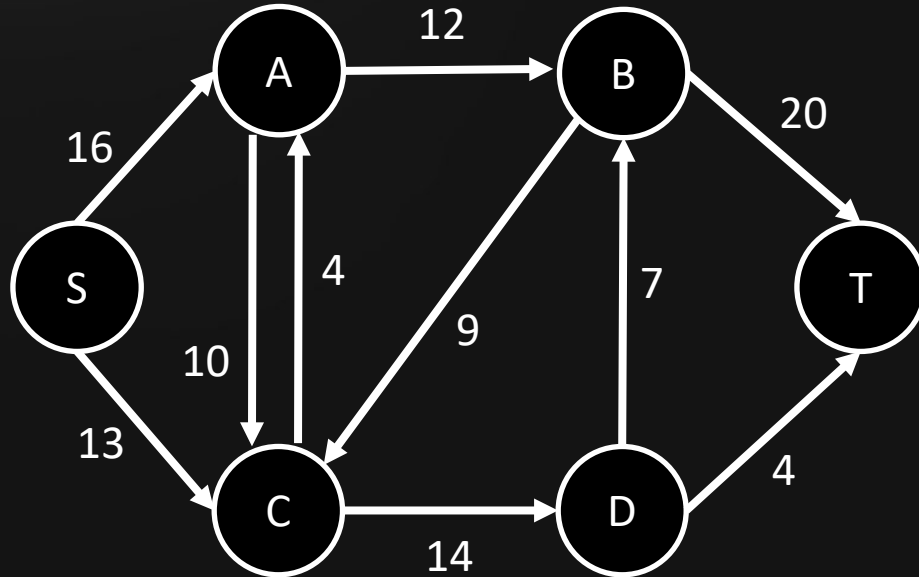


Network Flow Problem

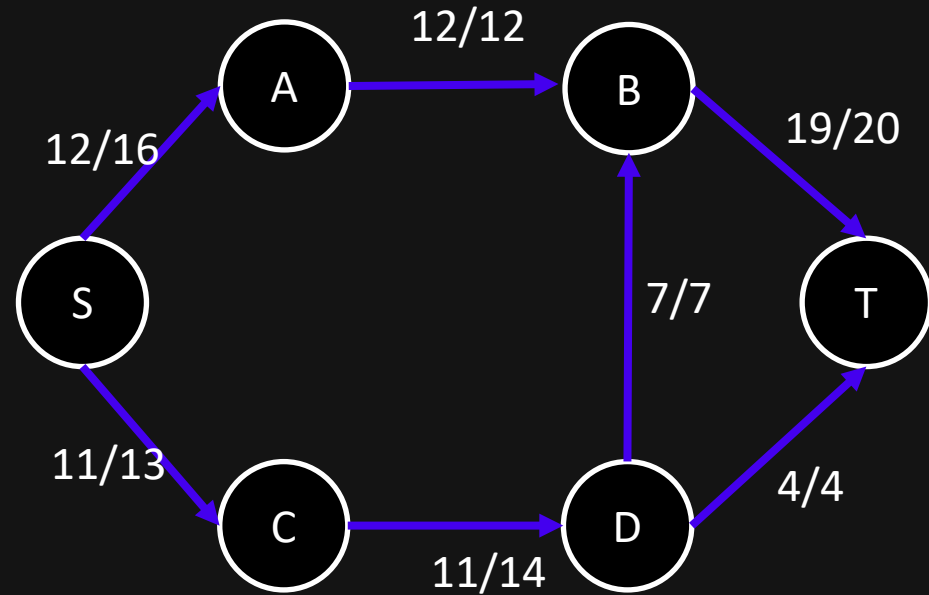
- **Definition:**
- Given a directed graph $G = (V, E)$, where each edge e is associated with its capacity $c(e) > 0$. Two special nodes s and sink t are given ($s \neq t$).
- **Problem:**
- Maximize the total amount of flow from s to t subject to two constraints
- Flow on edge e doesn't exceed $c(e)$
- For every node $v \neq s, t$, incoming flow is equal to outgoing flow

Network Flow Problem

- Capacities



Maximum flow: 23



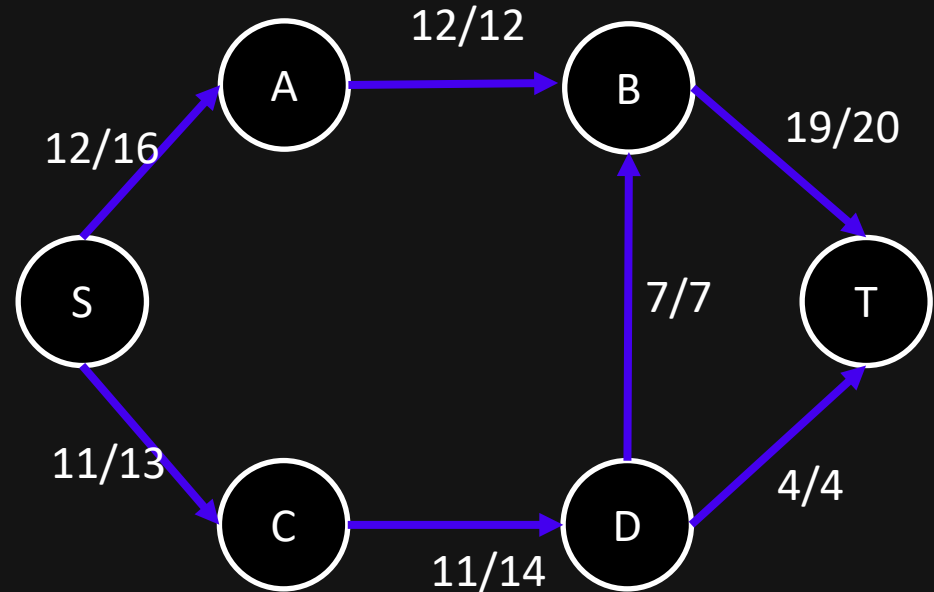


Network Flow Problem

- To remove edges from graph such that after removing the edges, there is no path from s to t .
- The cost of removing e is equal to its capacity $c(e)$
- The minimum cut problem is to find a cut with minimum total cost.

Network Flow Problem

- Any valid flow can be decomposed into flow paths and circulations
- $S \rightarrow A \rightarrow B \rightarrow T$: 12
- $S \rightarrow C \rightarrow D \rightarrow B \rightarrow T$: 7
- $S \rightarrow C \rightarrow D \rightarrow T$: 4





Ford Fulkerson Algorithm

- Ford-Fulkerson algorithm is a greedy approach for calculating the maximum possible flow in a network or a graph.
- It uses the **Depth First Search**.
- A term, **flow network**, is used to describe a network of vertices and edges with a source (S) and a sink (T). Each vertex, except S and T , can receive and send an equal amount of stuff through it. S can only send and T can only receive stuff.



Ford Fulkerson Algorithm

- We can visualize the understanding of the algorithm using a flow of liquid inside a network of pipes of different capacities.
- Each pipe has a certain capacity of liquid it can transfer at an instance.
- For this algorithm, we are going to find how much liquid can be flowed from the source to the sink at an instance using the network.



Ford Fulkerson Algorithm

- **Terminologies Used**
 - **Augmenting Path**
 - It is the path available in a flow network.
 - **Residual Graph**
 - It represents the flow network that has additional possible flow.
 - **Residual Capacity**
 - It is the capacity of the edge after subtracting the flow from the maximum capacity.

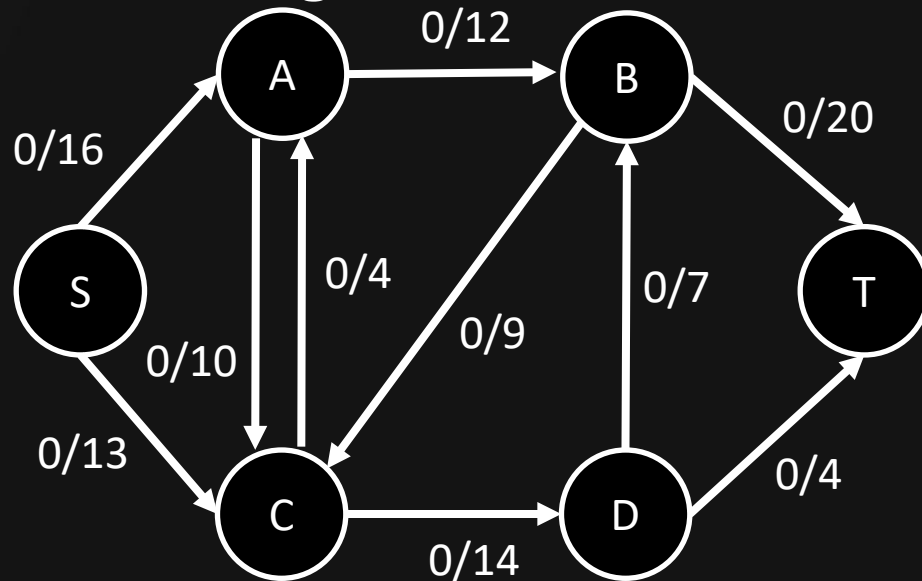


Ford Fulkerson Algorithm

- The algorithm follows:
 1. Initialize the flow in all the edges to 0.
 2. While there is an augmenting path between the source and the sink, add this path to the flow.
 3. Update the residual graph.

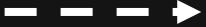
Ford Fulkerson Algorithm

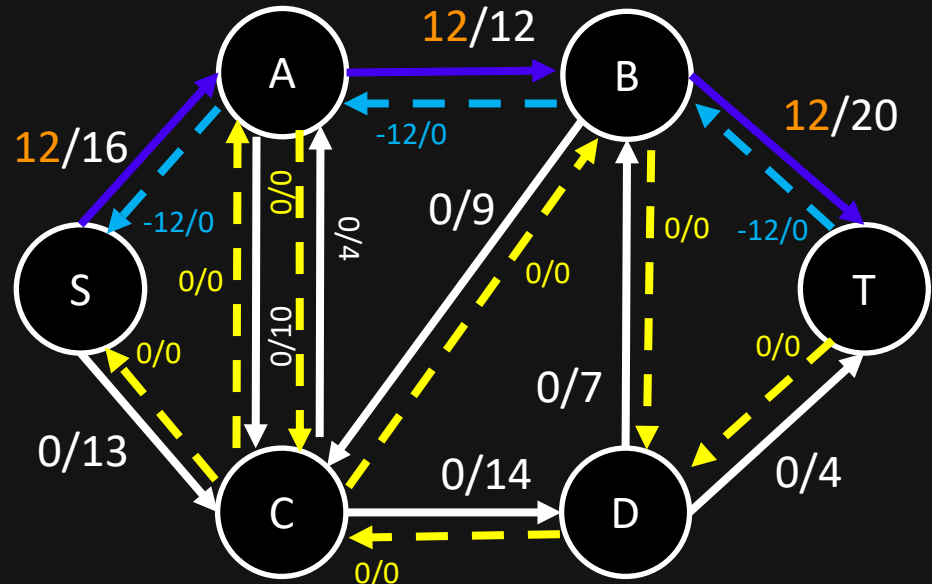
- Example:
- The flow of all the edges is 0 at the start.



Ford Fulkerson Algorithm

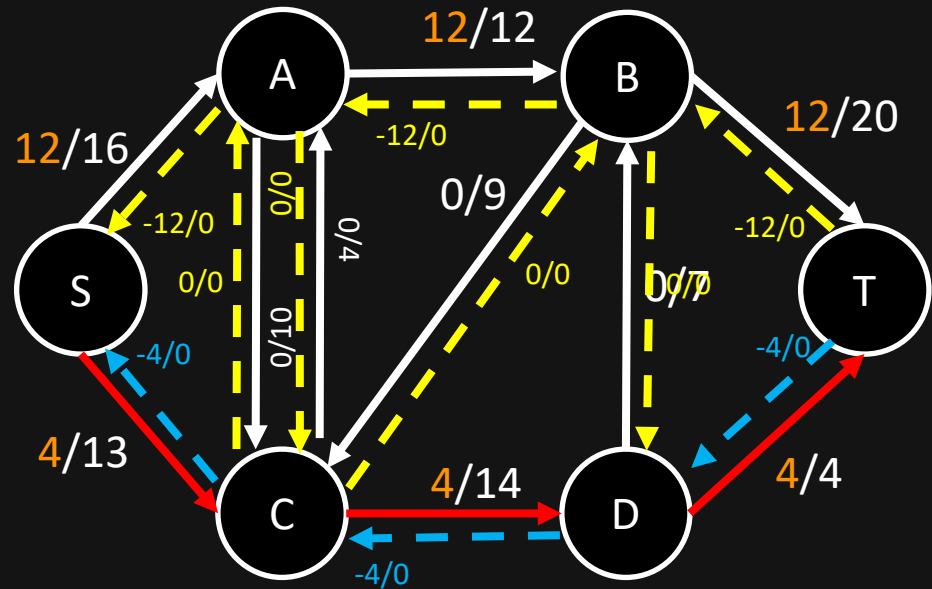
- Select any arbitrary path from S to T. We use $S \rightarrow A \rightarrow B \rightarrow T$
- Minimum capacity among the three edges is 12 for $A \rightarrow B$. Based on this the **flow/capacity** for each path is updated.

 Residual Path  Augmenting Path



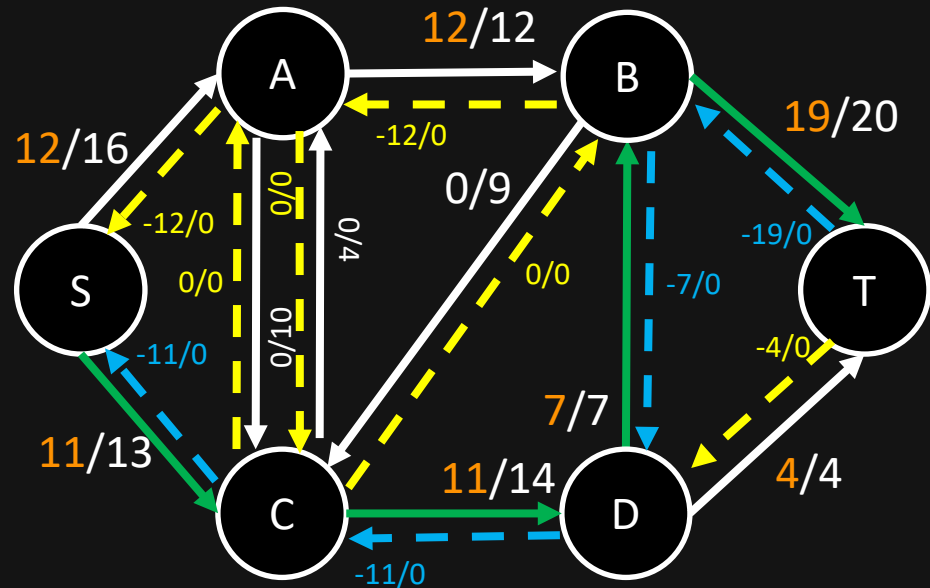
Ford Fulkerson Algorithm

- Select another from S to T. We use $S \rightarrow C \rightarrow D \rightarrow T$
- Minimum capacity among the three edges is 4 for $D \rightarrow T$. Based on this the **flow/capacity** for each path is updated.



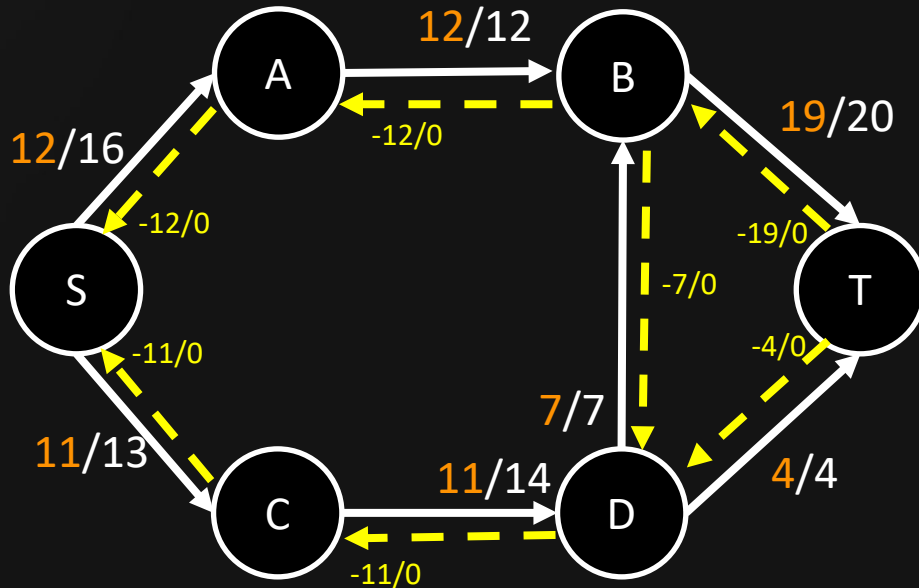
Ford Fulkerson Algorithm

- Select another from S to T. We use $S \rightarrow C \rightarrow D \rightarrow B \rightarrow T$
- Minimum capacity among edges are
- $S \rightarrow C$: 9
- $C \rightarrow D$: 10
- $D \rightarrow B$: 7
- $B \rightarrow T$: 8
- The minimum residual capacity among these edge is 7 for $D \rightarrow B$
- Based on this the **flow/capacity** for each path is updated.



Ford Fulkerson Algorithm

- Adding all the flows = $12+7+4 = 23$, which is the maximum possible flow on the flow network.





Ford Fulkerson Algorithm

- Algorithm

FORD-FULKERSON METHOD (G, s, t)

$flow = 0$

FOR each edge (u, v) in G :

$flow(u, v) = 0$

WHILE there is a path, p , from $s \rightarrow t$ in residual network G_f :

$residual_capacity(p) = \min(residual_capacity(u, v) : \text{for } (u, v) \text{ in } p)$

$flow = flow + residual_capacity(p)$

FOR each edge (u, v) in p :

IF (u, v) is a forward edge:

$flow(u, v) = flow(u, v) + residual_capacity(p)$

ELSE:

$flow(u, v) = flow(u, v) - residual_capacity(p)$

RETURN flow



Ford Fulkerson Algorithm

- Ford Fulkerson algorithm has **time complexity $O(|E| * f)$** , where $|E|$ is the number of edges and ' f ' is the maximum flow of a network because it depends on the maximum flow of the network. Edmonds Karp algorithm has time complexity **$O(|V| * E^2)$** , where E is the number of edges and V is the number of vertices.
- **The space complexity** of the Edmonds Karp algorithm is **$O(V)$** in the worst case. Creating a queue and inserting each node in it takes **$O(V)$** extra space.



Ford Fulkerson Algorithm

- Residual edges have capacity of 0, it is to think of remaining capacity of an edge e (residual or not) as: $e.\text{capacity} - e.\text{flow}$, this ensures that remaining capacity of an edge is always non negative.
- **Applications:**
 - Water distribution pipeline
 - Bipartite matching problem
 - Circulation with demands



Edmonds Karp Algorithm

- Ford-Fulkerson method is a common technique used to find the maximum flow.
- Ford-Fulkerson method use **Depth First Search**, and repeatedly find augmenting paths from $S \rightarrow T$ in the flow graph, augment the flow and repeat until no more paths exist.
- **Ford Fulkerson does not specify how to actually find augmenting paths.**



Edmonds Karp Algorithm

- Ford-Fulkerson method using a DFS to find augmenting path takes $O(Ef)$, where E is number of edge and f is maximum flow.
- Edmonds-Karp Algorithm uses a Breath First Search (BFS) to find augmenting paths which yields an arguably better time complexity of $O(VE^2)$.
- The major difference in this approach is that the time complexity no longer depends on the capacity value of any edge.

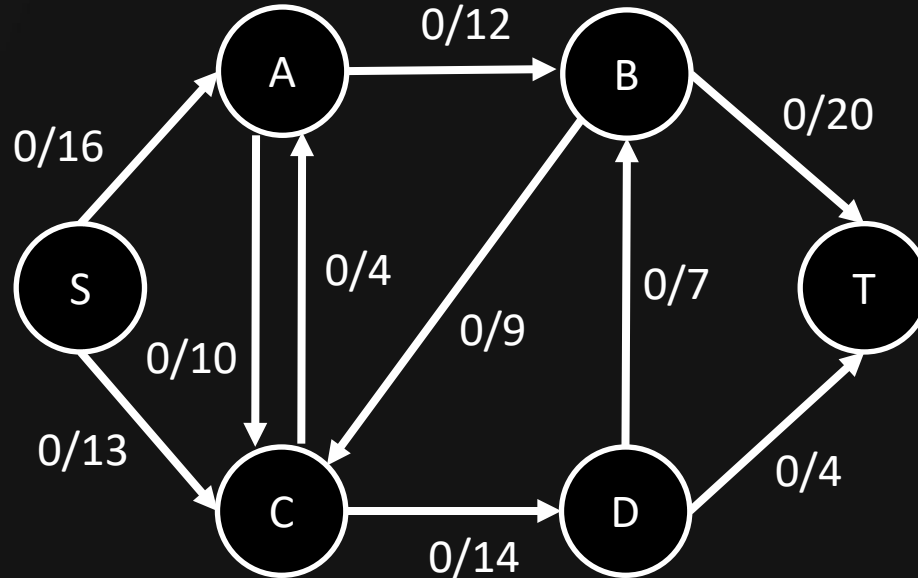


Edmonds Karp Algorithm

- **Edmonds-Karp Algorithm** is method of augmentation which repeatedly finds the shortest augmenting path from $S \rightarrow T$ in terms of the number of edges used each iterations.
- Using BFS to find augmenting paths ensures that the shortest path from $S \rightarrow T$ is found every iterations.

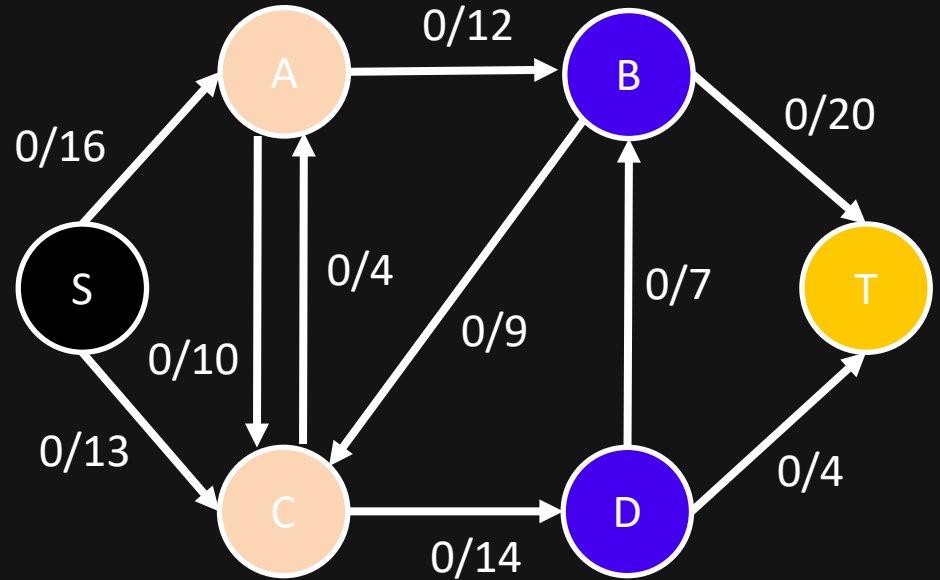
Edmonds Karp Algorithm

- Example:
- The flow of all the edges is 0 at the start.



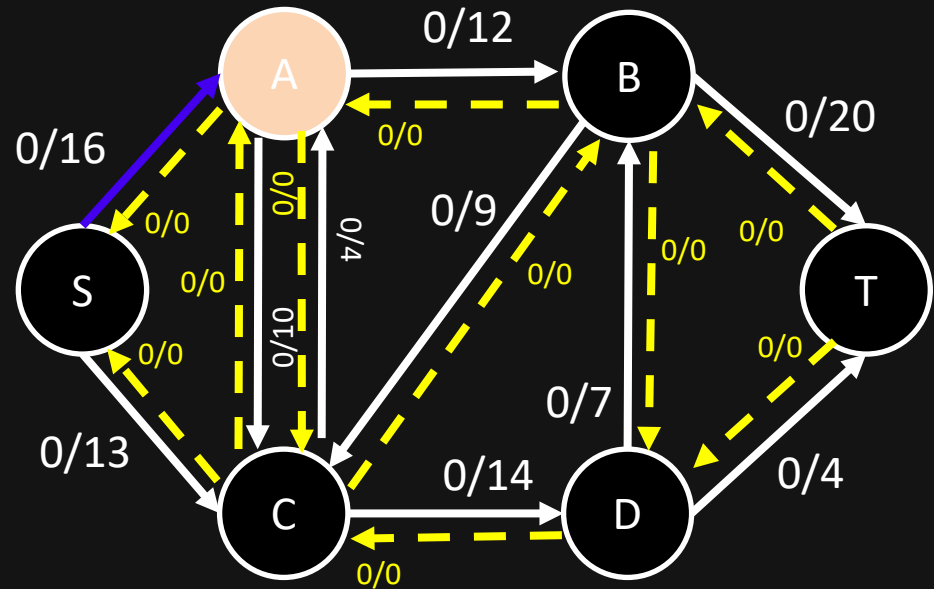
Edmonds Karp Algorithm

- Select the nodes in BFS, so we enqueue **A** then **C**,
- After that **B** and **D**
- Finally we reach Sink node **T**.



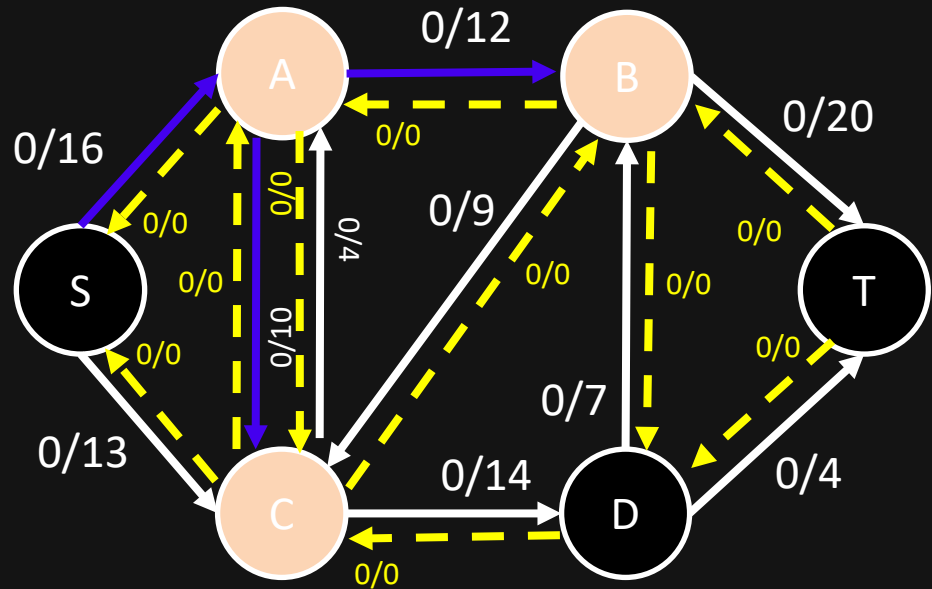
Edmonds Karp Algorithm

- In Queue we have A,C,B,D,T.
- We start with A and try to find the shortest path that can reach the sink node first.



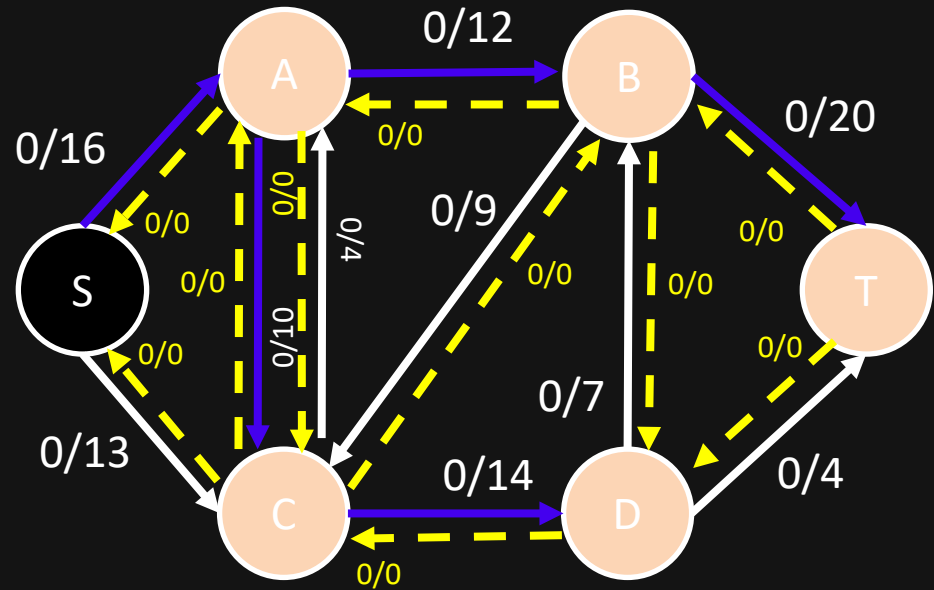
Edmonds Karp Algorithm

- After we reach A, we try to reach neighbor nodes.



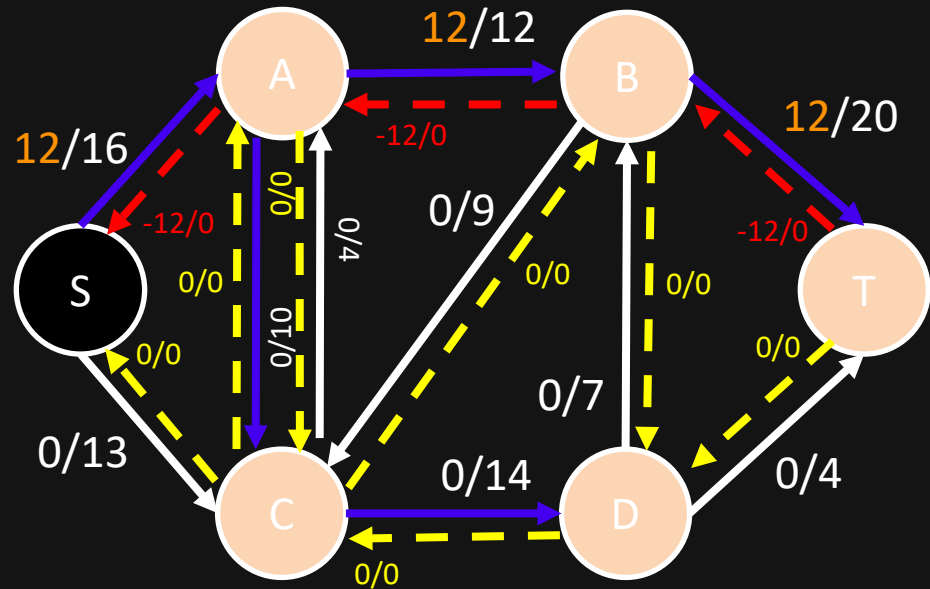
Edmonds Karp Algorithm

- Now we try to reach next neighbor node.
- We should stop if we reached the sink node T.



Edmonds Karp Algorithm

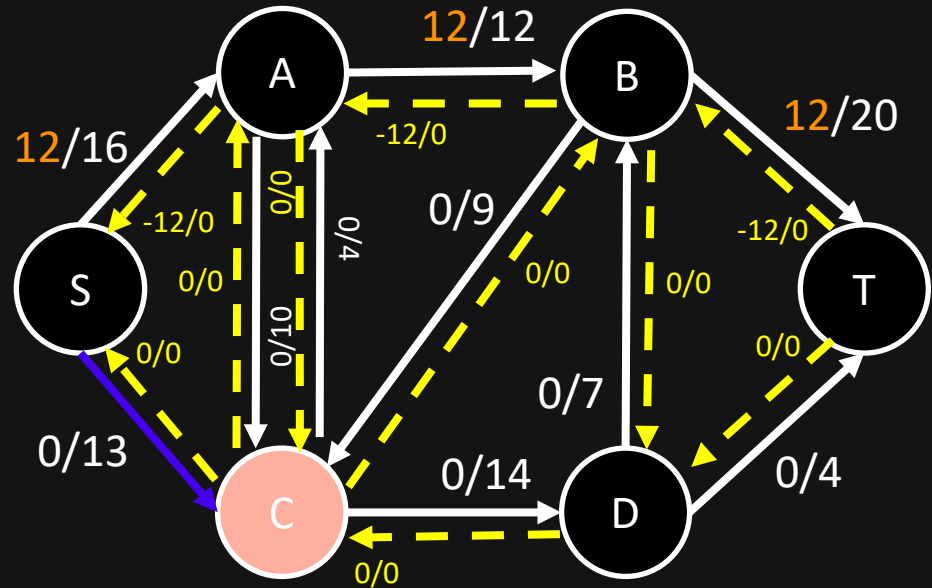
- We reached the Sink T, with shortest path, now we update the residual graph.



---> Residual Path
——> Augmenting Path

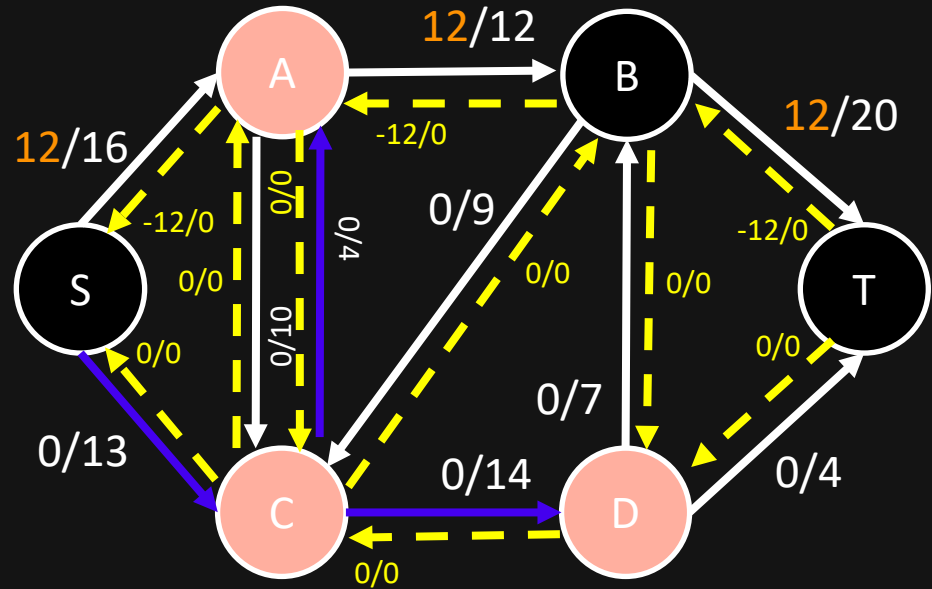
Edmonds Karp Algorithm

- In Queue we have C,B,D,T.
- Now we start with C and try to find the shortest path that can reach the sink node first.



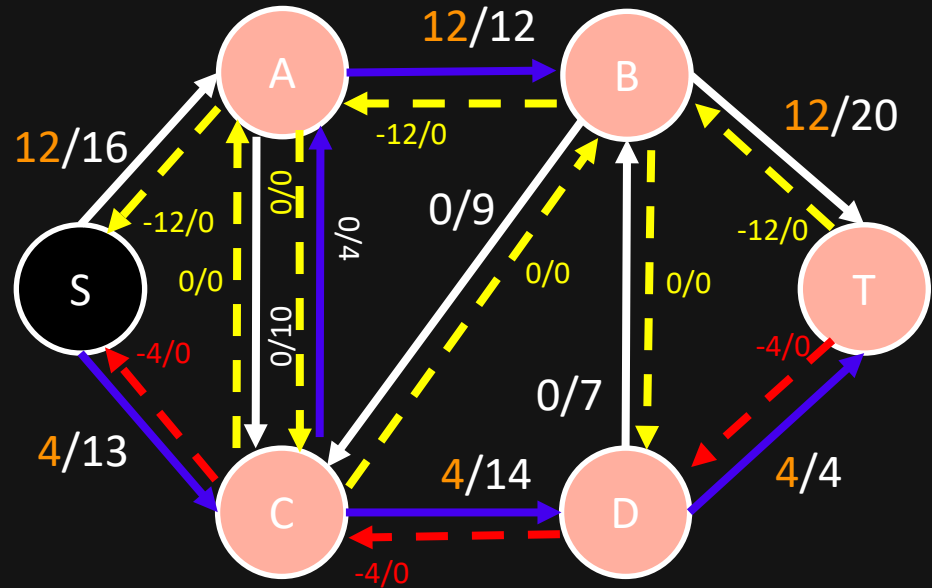
Edmonds Karp Algorithm

- Now we try to reach next neighbor node.
- We should stop if we reached the sink node T.



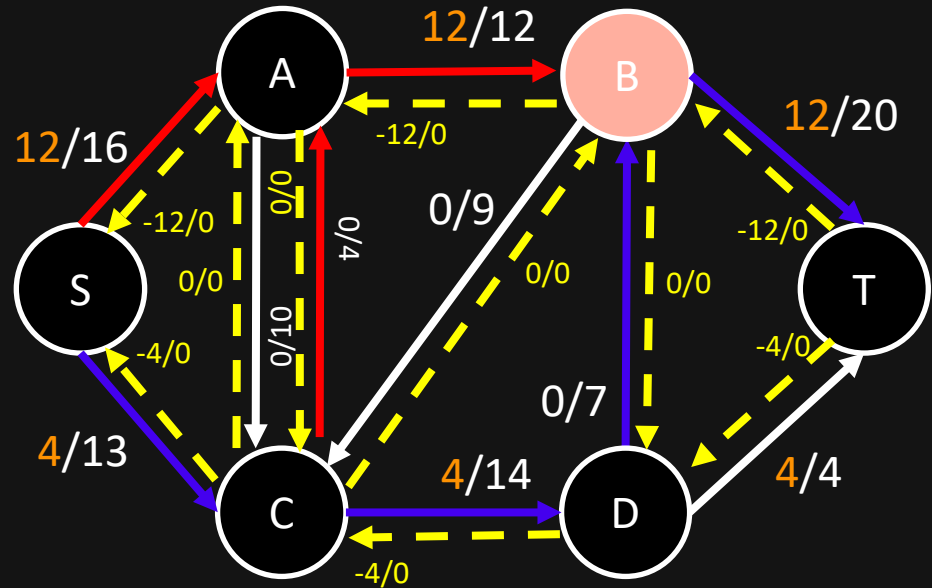
Edmonds Karp Algorithm

- We reached the Sink T, with shortest path, now we update the residual graph.



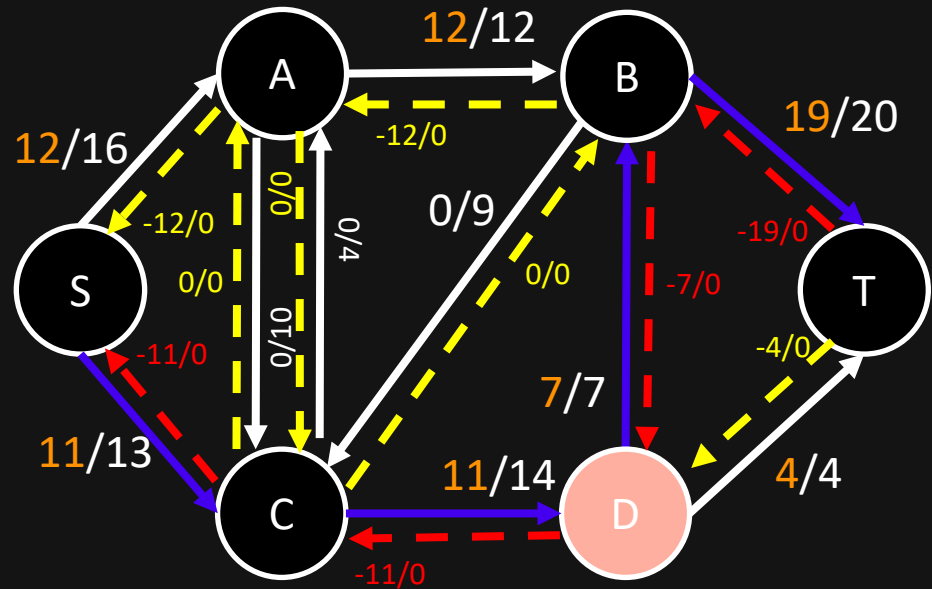
Edmonds Karp Algorithm

- In Queue we have B,D,T.
- We see any path to reach sink using B.
- The Path $S \rightarrow A \rightarrow B$ and $S \rightarrow C \rightarrow A \rightarrow B$ cant be used since $A \rightarrow B$ capacity is fully used.
- Hence we use $S \rightarrow C \rightarrow D \rightarrow B \rightarrow T$ to reach T.



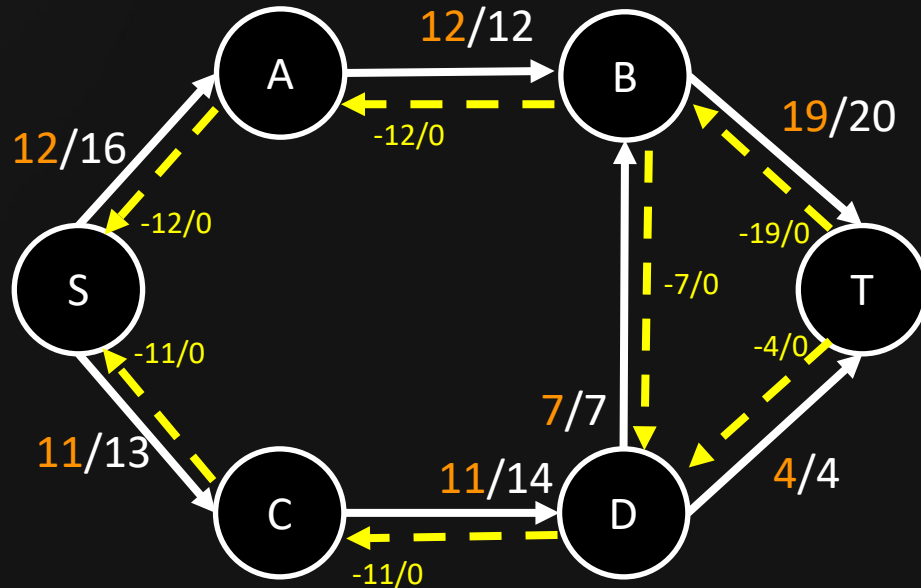
Edmonds Karp Algorithm

- Update the graph
- Next node is D as you can see edge from D, $D \rightarrow B$ and $D \rightarrow T$ is fully used. Hence no more path to reach the sink node T from D.



Edmonds Karp Algorithm

- Adding all the flows = $12+7+4 = 23$, which is the maximum possible flow on the flow network.





Edmonds Karp Algorithm

Algorithm Pseudocode

Inputs: capacityMatrix[n x n],
adjMatrix[n x n], src, target

Output: maxflow

The Edmonds-Karp:

maxFlow = 0

residualMatrix[n x n]

WHILE true:

 min, augmentPath =
 BFS(capacityMatrix,
 adjMatrix, source, target,

residualMatrix)

IF min = 0:

BREAK

 maxFlow = maxFlow + min

 v = target

WHILE v != src:

 u = P[v]

 residualMatrix[u,v] -= min

 residualMatrix[v,u] += min

 v = u

RETURN maxFlow



Edmonds Karp Algorithm

- This algorithm code starts with the maximum flow initially set to 0.
- The while loop executes until there are no more augmenting paths. Within the while loop, we call BFS to find the shortest path from source to sink and the minimum residual capacity along that path, $\min$.
- We then walk the augmenting path from target to source. Using the minimum residual capacity, we reduce all residual capacities on the augmenting path by $\min$ and increase the residual capacities on the reverse edges (representing the flow).



Edmonds Karp Algorithm

- Complexity Analysis
- The Edmonds-Karp algorithm runs in $O(VE^2)$
- In each iteration of the algorithm, the shortest path (BFS) between the source and all other vertices must increase monotonically. We need to prove that one iteration of the Edmonds-Karp algorithm is bounded by $O(E)$
- . We then need to prove that the number of iterations of the algorithm to find the maximum flow of a network is bounded by $O(VE)$ iterations. Proving these two parts implies that the Edmonds-Karp algorithm is bounded by $O(VE^2)$.



Push Re-label Algorithm

- Idea
- Push as much flow throughout the residual graph by using multiple paths in a single iteration
- Definition
- A blocking flow g in a flow network G is a feasible flow such that for every $S \rightarrow T$ path P of G some edge is saturated by g . ($f_e = u_e$)
- That is a blocking flow zero's out an edge of every possible $S \rightarrow T$ path.



Push Re-label Algorithm

- Let f be a flow in the graph G . The following are equivalent:
 - f is a maximum flow of G
 - There is an $S \rightarrow T$ cut (A, B) of G such that the value of f equals the capacity of (A, B)
 - There is no $S \rightarrow T$ path with positive residual flow in the residual network G_f .
- If any of these statements is true then they all are true!



Push Re-label Algorithm

- The algorithms you are most familiar with have been derived by trying to find an algorithm that maintains feasibility (satisfies constraints) and then algorithm that works to optimality.
- The next algorithm will be developed by satisfying the optimality condition (no $S \rightarrow T$ path in the residual graph) at every stage and working towards satisfying the constraints.
- The algorithm will satisfy the capacity constraints $f_e \leq u_e$ while working toward satisfying the conservation constraints flow into $v_{flow\ in} = v_{flow\ out}$



Push Re-label Algorithm

- Idea: Push as much flow as possible ignoring conservation constraints
- Fill up the edges all the way to the sink
- If there is excess flow into a node, then the excess is sent to downstream nodes respecting capacity constraints.
- Once all the possible flow reaches the sink, excess flow is then routed back toward the source, so that conservation constraints are restored.
 - Some of that flow may be sent to another vertex that will use different paths.
 - Eventually all the excess flow is routed back to the sink.



Push Re-label Algorithm

- **Intuition: Conceptualize the residual graph as if it is in 3D**
- Edges are like water pipes.
- $h(v)$ is the height of the node v
 - S is at height n , and t is at height 0
 - Cannot flow uphill at all or downhill if the hill is too steep
- As the algorithm proceeds it changes the levels of the nodes to allow any excess to flow downhill (towards t)
- Notice that in the residual graph of this network there is no $S \rightarrow T$ path – this is the optimality criteria.
- Once the conservation constraints are met all, then we have a feasible flow.



Push Re-label Algorithm

- Preflow
- An $S \rightarrow T$ **preflow** (preflow for short) is a function $f: E \rightarrow \mathbb{R}^+$ such that it satisfies:
- For each $e \in E$, we have $0 \leq f_e \leq c_e$ capacity conditions
- For each $v \in V, v \neq s$ we have $\sum_{e \text{ into } v} f(e) \geq \sum_{e \text{ out of } v} f(e)$ at least as much flow entering as leaving
- The excess at v will be $\alpha_f(v) = \sum_{e \text{ into } v} f(e) - \sum_{e \text{ out of } v} f(e)$
- Residual graph will work as before.



Push Re-label Algorithm

- Preflow-Push Algorithm
- Maintains a preflow and works to convert to a flow.
- A labeling and a preflow f are compatible if
- Initial conditions: $h(s) = n, h(t) = 0$
- Steepness conditions: $\forall (v, w) \in E_f$ of the residual graph $h(v) \leq h(w) + 1$
- Notice that in the residual graph of this network there is no $S \rightarrow T$ path (proof: any $S \rightarrow T$ path would be of length n)
- Prop: if an $S \rightarrow T$ flow is compatible with the labeling h then it is maximum value.



Push Re-label Algorithm

- **High Level: Preflow-Push Algorithm**
- Augmenting path algorithms maintain feasibility at every stage, and gradually strive towards optimality.
- Pre-flow push algorithm initially pushes the maximum possible amount of flow, while maintaining optimality condition, and strives towards feasibility.
 - Initially, flow in every edge originating from the source is set to the link capacity.
 - Height label of every node (except source) is initially set to 0.
 - Heights are slowly raised to enable flow toward the sink
 - Once all the paths to the sink are saturated, excess flows back to the source.



Push Re-label Algorithm

- Pre-flow – relax conservation constraints, but keep the capacity constraints
- Define the excess at $v \rightarrow \alpha_f(v) = \text{flow in } v - \text{flow out } v$.

Push($f, h, (v, w)$)

Let $P = \min\{\alpha_f(v), u_{(v,w)} - f_{(v,w)}\}$

IF (v, w) is forward edge in G_f

$P = \min\{\alpha_f(v), u_{(v,w)} - f_{(v,w)}\}$ increase $f_{(v,w)}$ by P

IF (v, w) is backward edge in G_f

$P = \min\{\alpha_f(v), f_{(w,v)}\}$ decrease $f_{(w,v)}$ by P



Push Re-label Algorithm

Relabel($f, h, (v, w)$)

increase $h(v)$ by 1

RETURN (f, h)

Preflow-Push

$h(v) = 0, \forall v \neq s, h(s) = n = |V|$

$f_e = c_e, \forall e = (s, v); \text{ else } f_e = 0$

WHILE $\exists v \neq t \text{ and } \alpha_f(v) > 0$

let v be a node with excess at maximum height

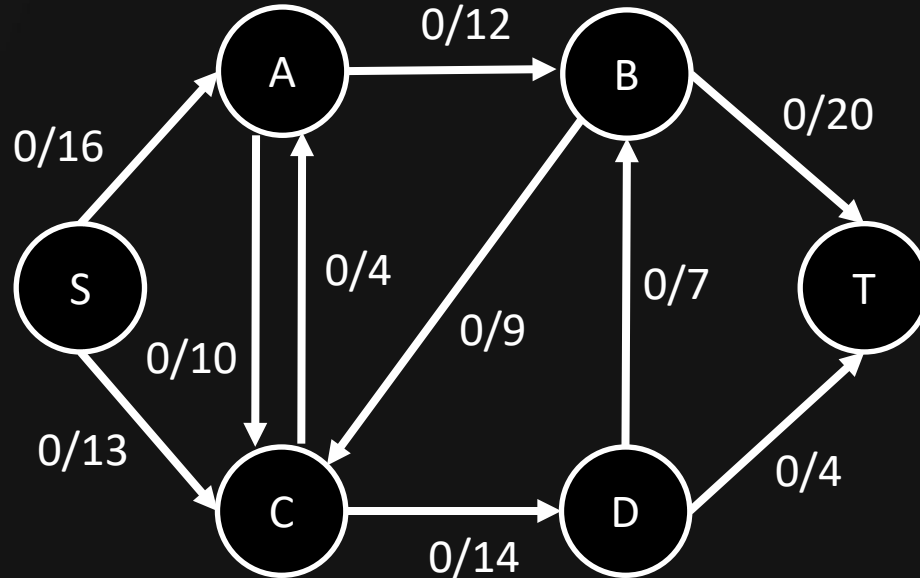
IF w such that preconditions hold for $Push(v)$

THEN $Push(f, h, (v, w))$

ELSE $Relabel(f, h, v)$

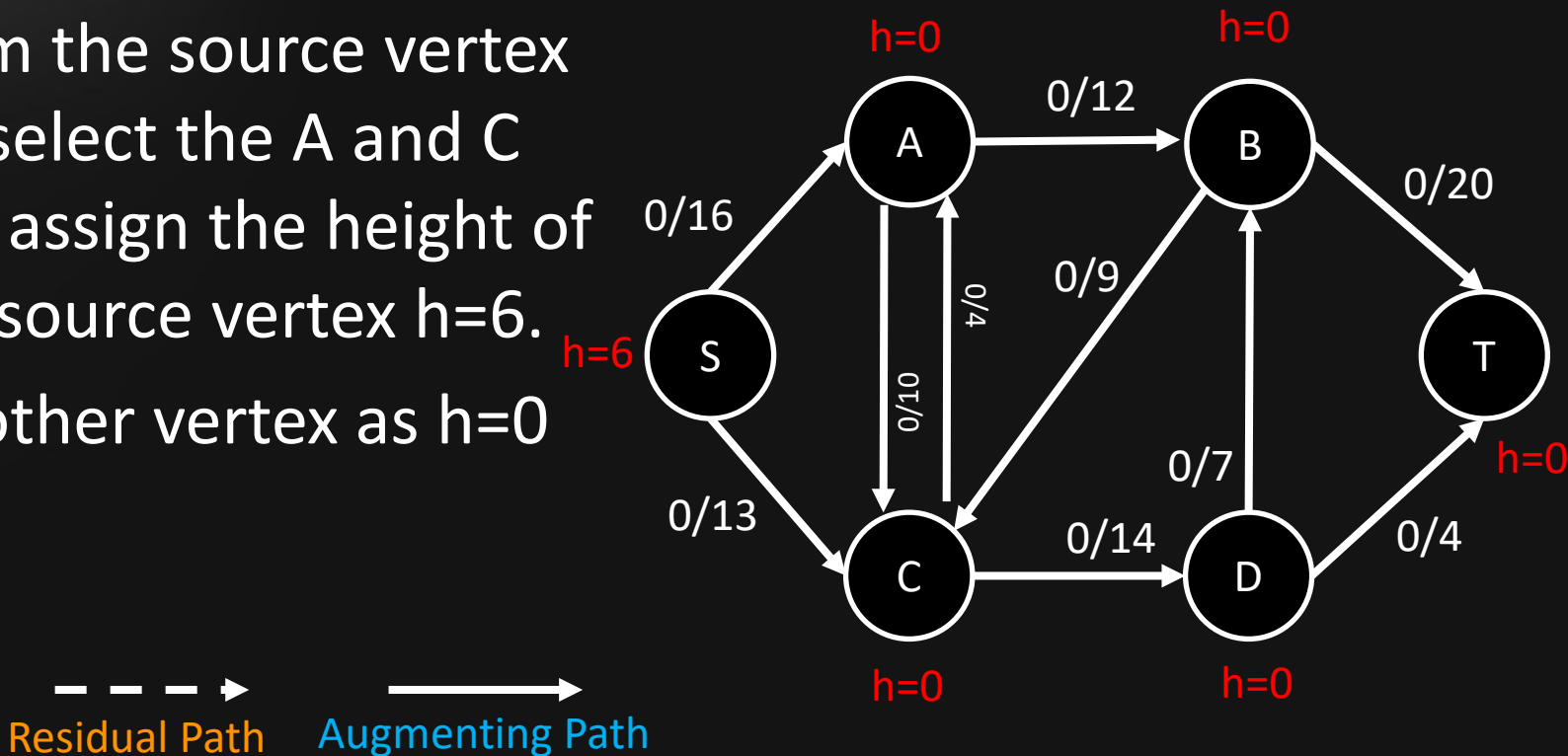
Push Re-label Algorithm

- Example:
- The flow of all the edges is 0 at the start.



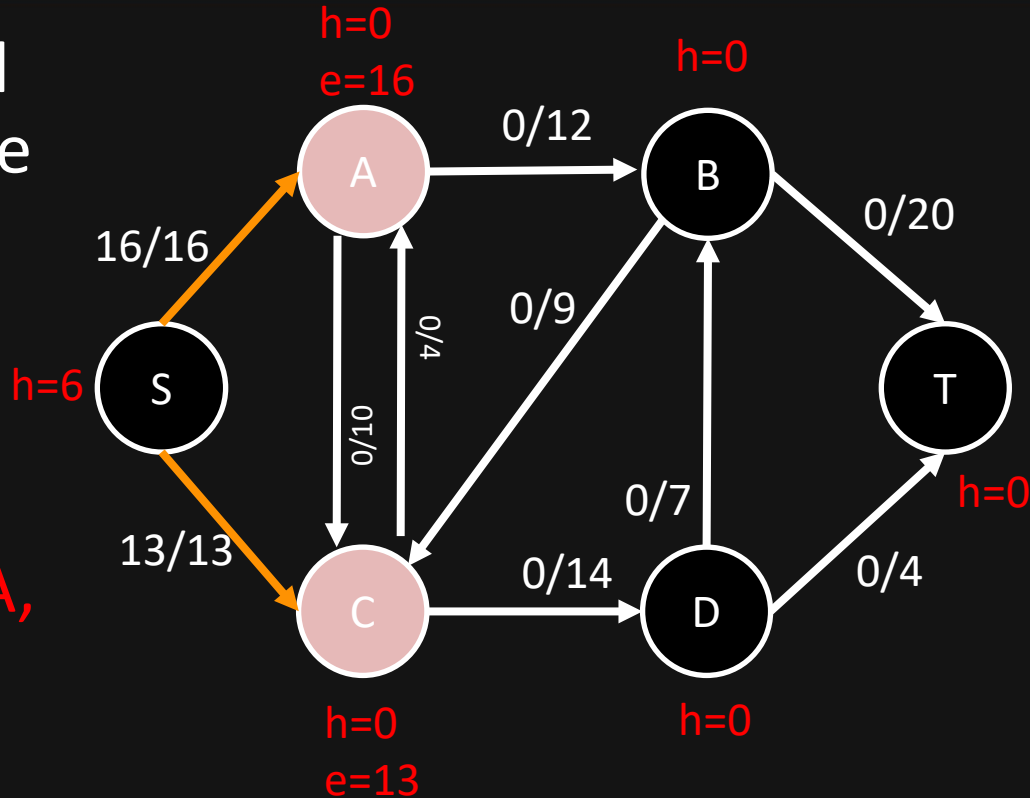
Push Re-label Algorithm

- From the source vertex we select the A and C and assign the height of the source vertex $h=6$.
- All other vertex as $h=0$



Push Re-label Algorithm

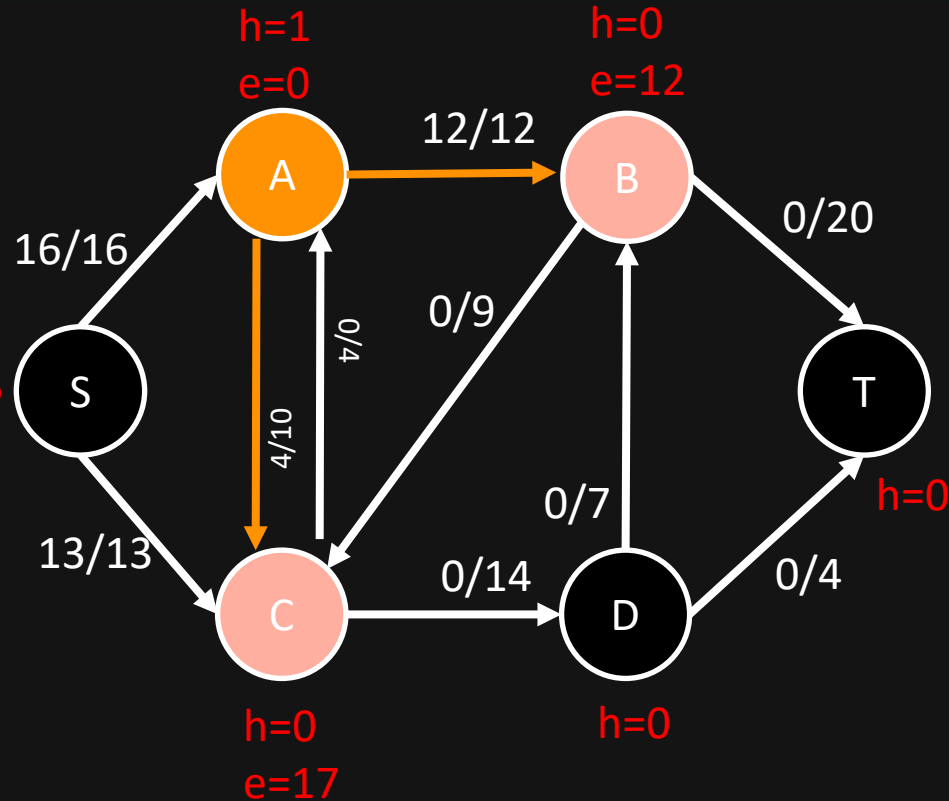
- We select vertex A and C which adjacent to the source vertex.
- We update the e to their maximum capacity.
- **Active Vertex Queue: A, C**



Push Re-label Algorithm

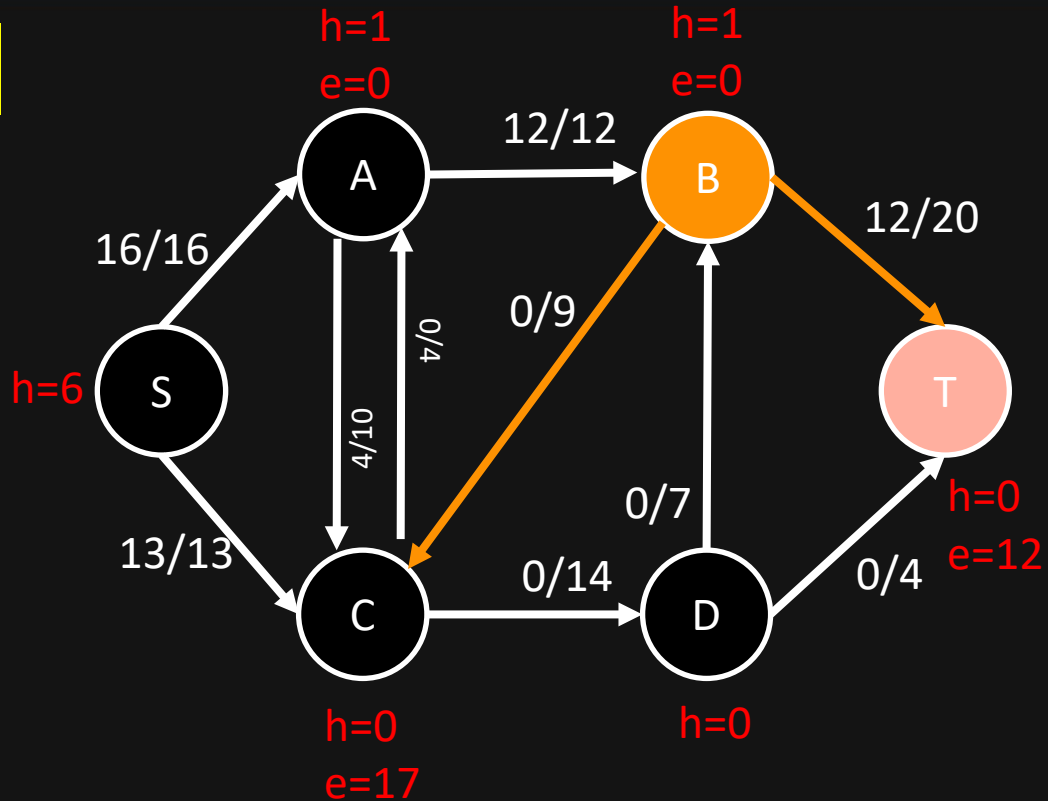
- Next we go to the adjacent vertex of A, i.e is B and C.
- e value is updated for B and remaining capacity is sent back to A and height of A is updated.

• Active Vertex: **Current**
=A, B, C



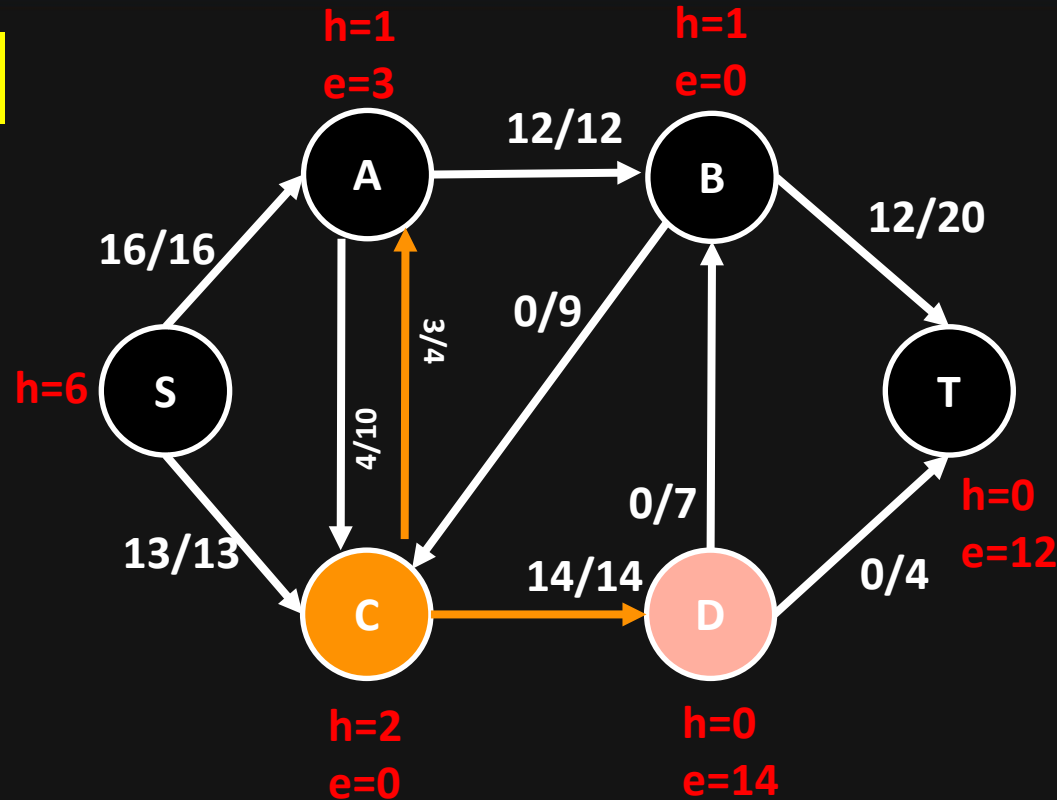
Push Re-label Algorithm

- Active Vertex: **Current = B, T, C.**
- Height of B is update and capacity for T is updated.
- Next Active Vertex: **Current = T, C.**
- T is processed.



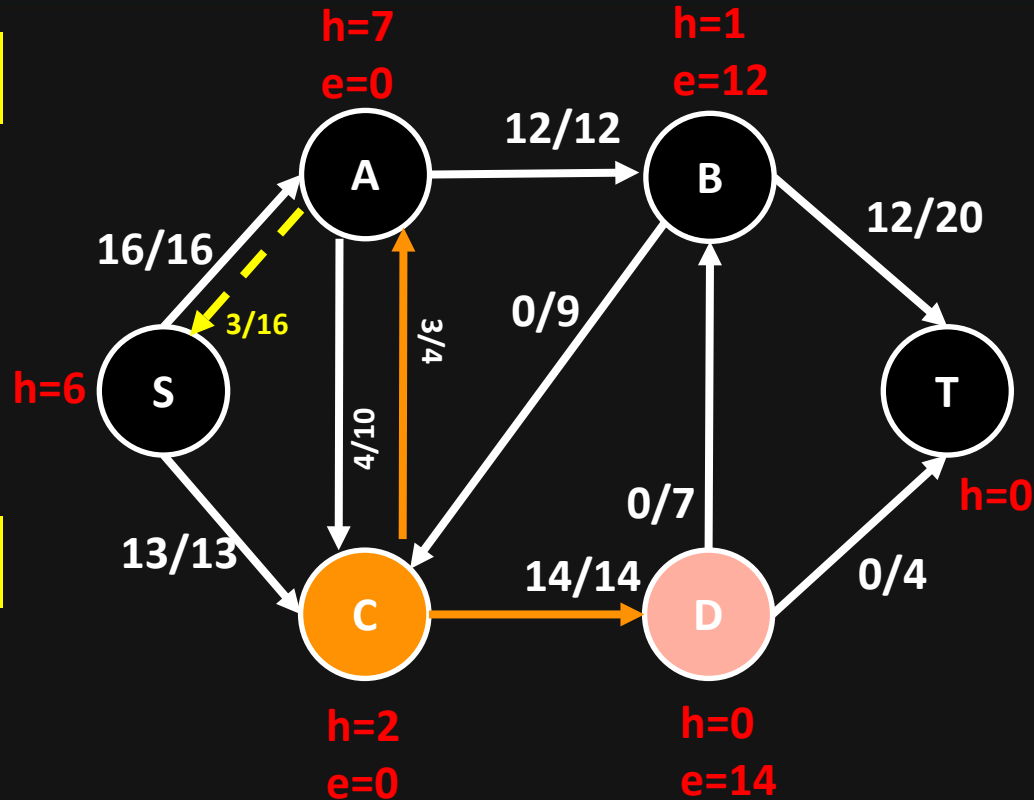
Push Re-label Algorithm

- Active Vertex: **Current**
=C



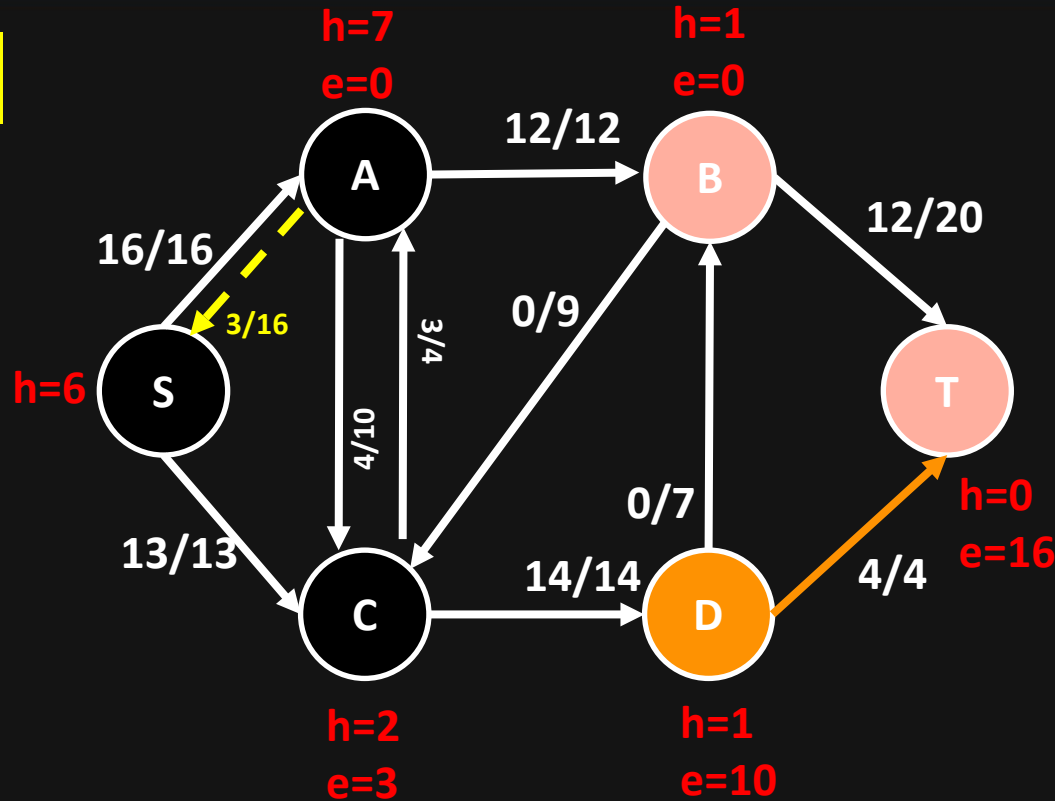
Push Re-label Algorithm

- Active Vertex: **Current = C**
- Excess capacity is sent back to S from A and height is updated.
- Active Vertex: **Current = C, D**



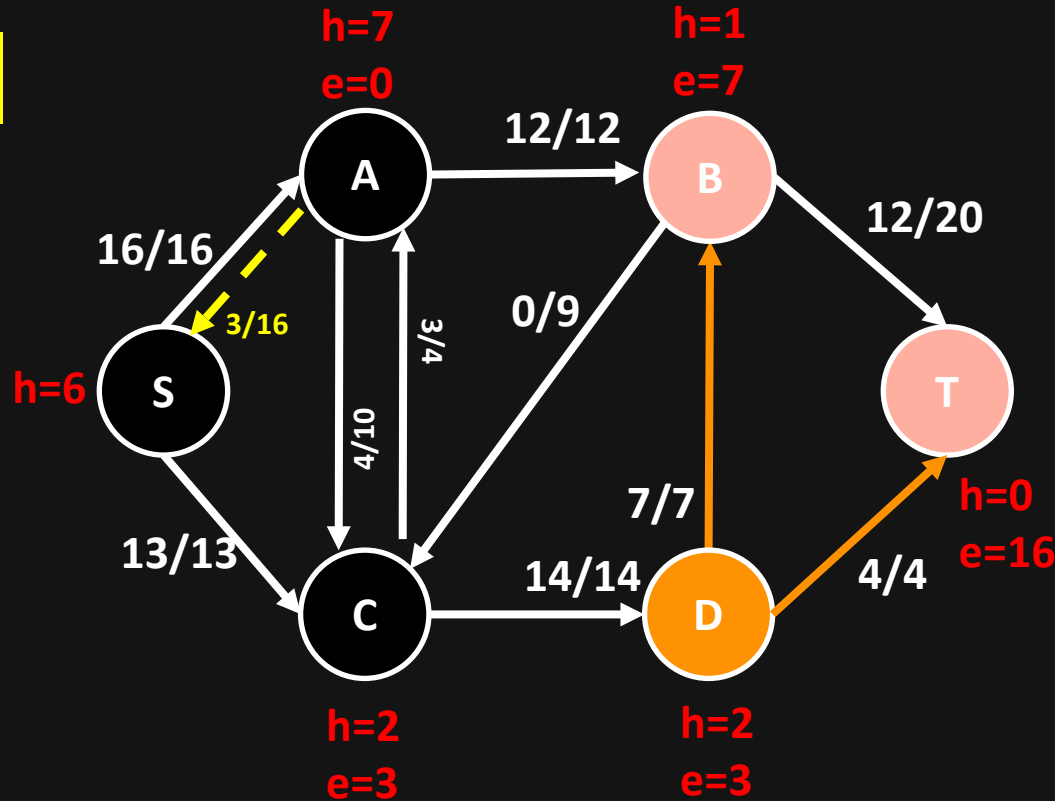
Push Re-label Algorithm

- Active Vertex: **Current**
=D, B, T



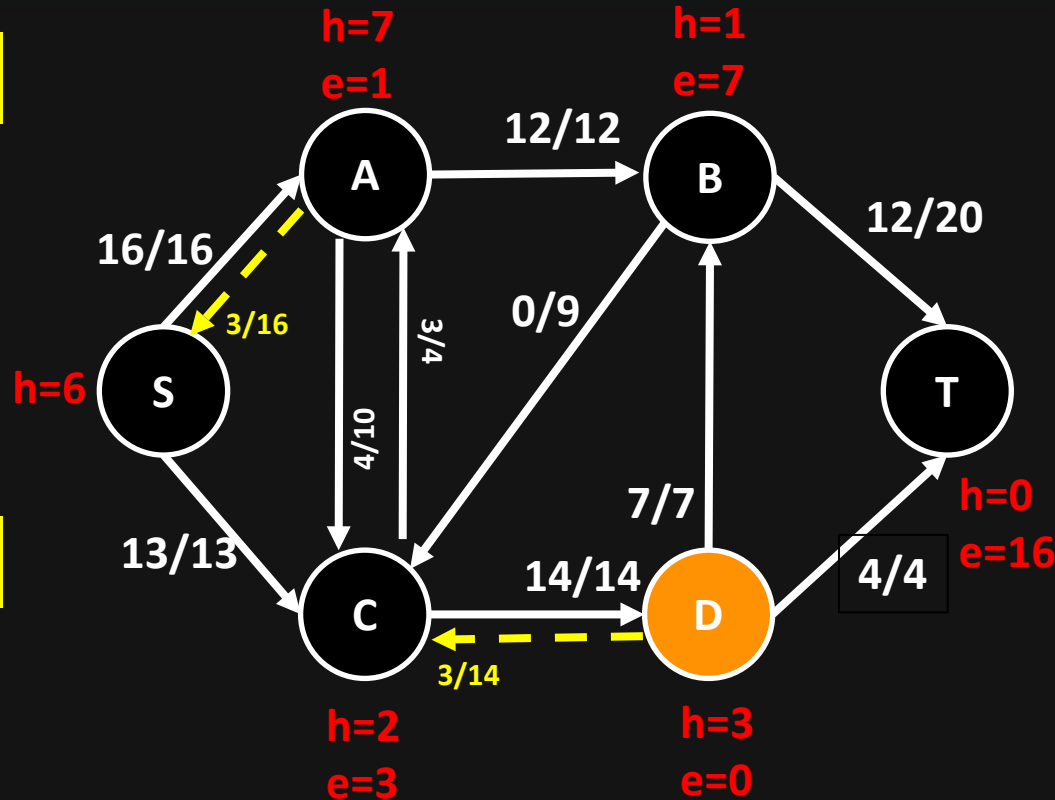
Push Re-label Algorithm

- Active Vertex: **Current**
=D, B, T



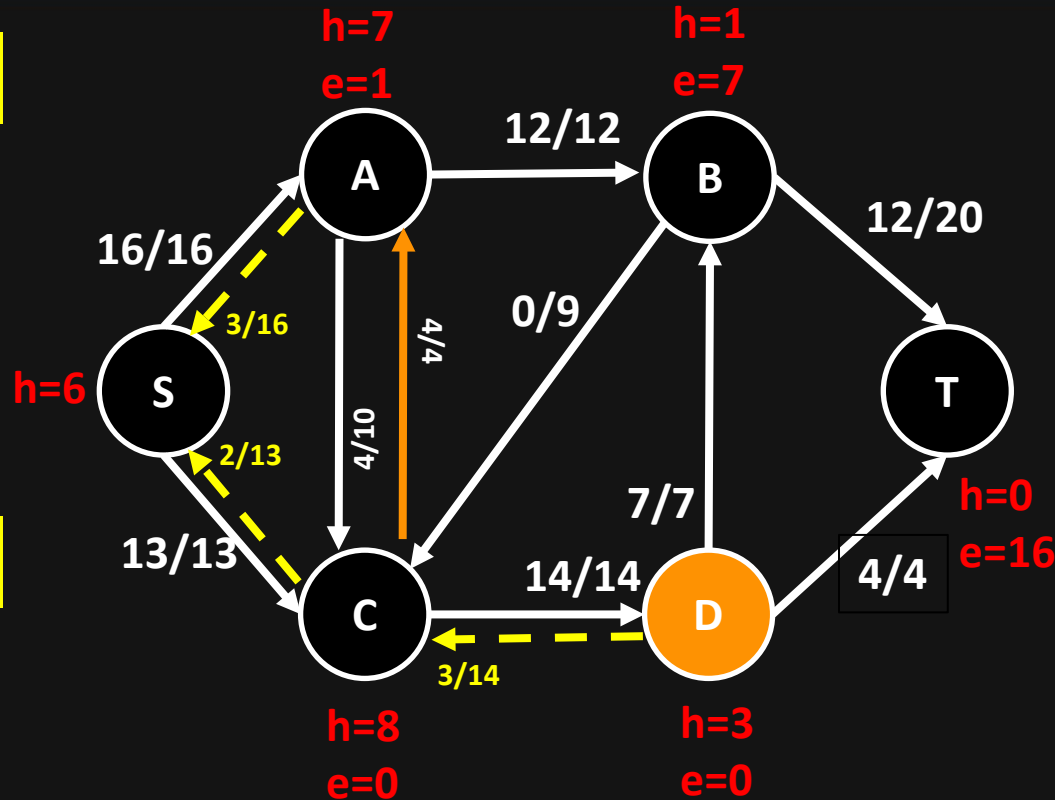
Push Re-label Algorithm

- Active Vertex: **Current = D, B, T**
- Excess capacity of D is sent back to C and height is updated.
- Active Vertex: **Current = B, T**



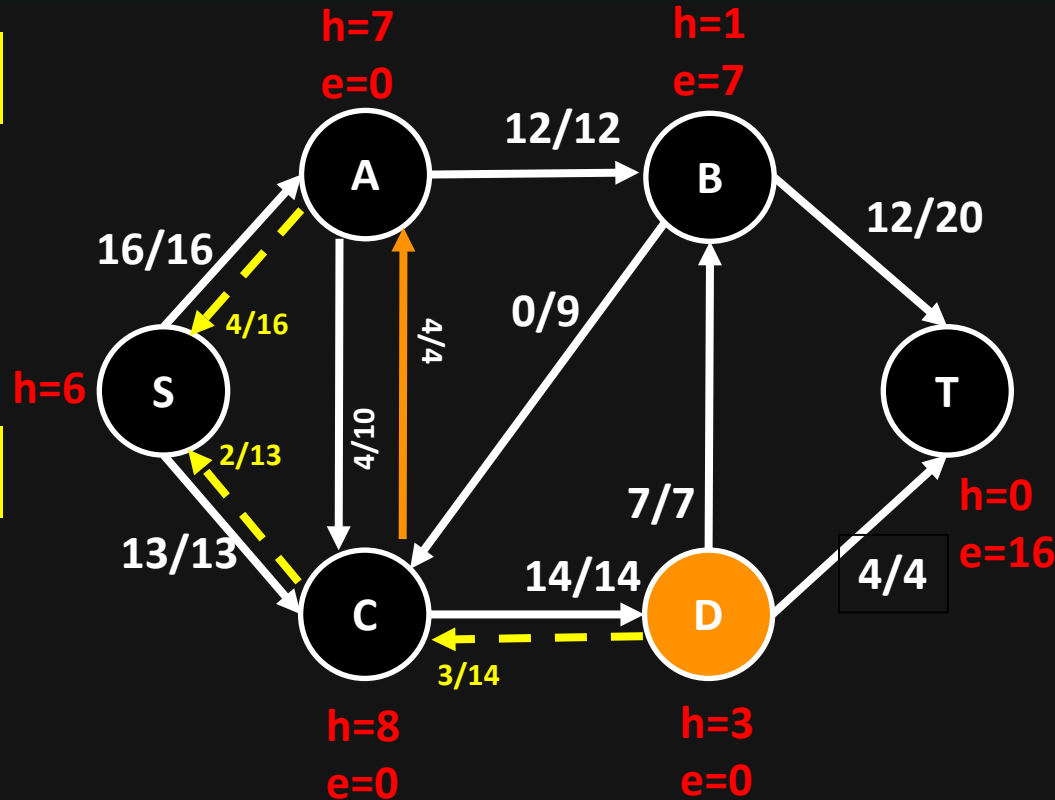
Push Re-label Algorithm

- Active Vertex: **Current = D, B, T**
- Excess capacity of C is sent back to S & A, height is updated.
- Active Vertex: **Current = B, T**



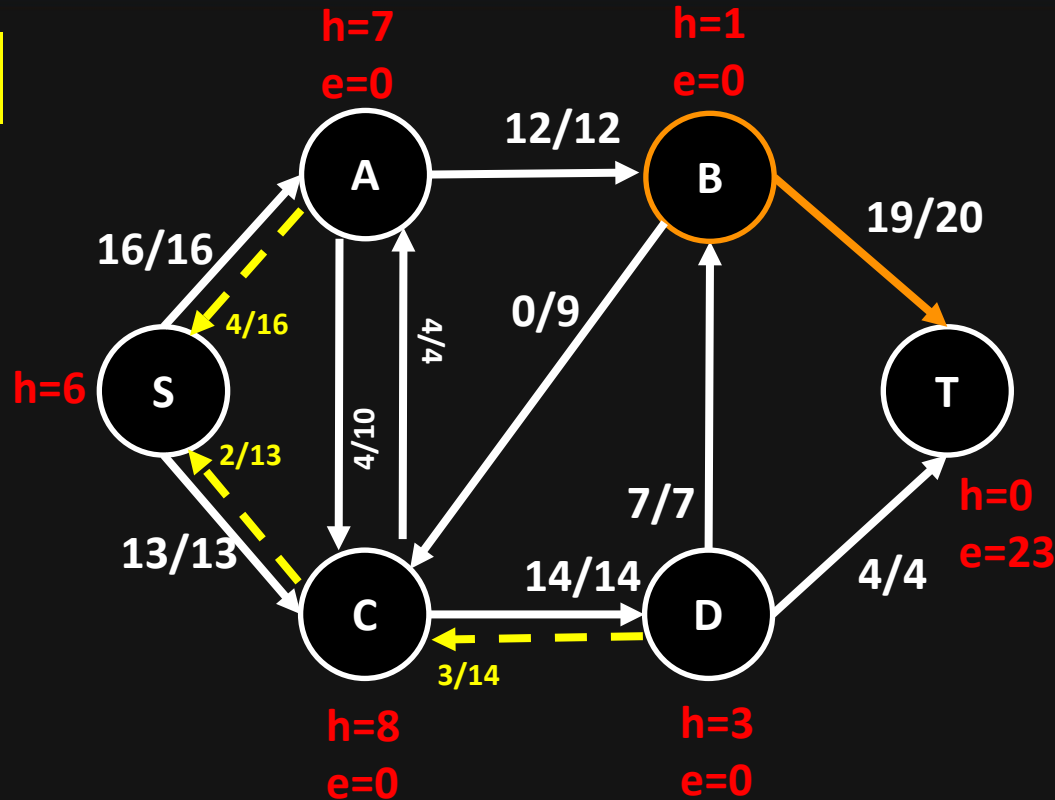
Push Re-label Algorithm

- Active Vertex: **Current = D, B, T**
- Excess capacity of A is sent back to S.
- Active Vertex: **Current = B, T**



Push Re-label Algorithm

- Active Vertex: **Current**
=B, T
- Next T is sink.
- So Max flow is $e = 23$





Push Re-label Algorithm

- Need to maintain some invariants to structure the algorithm
- Invariant “no $S \rightarrow T$ path in the residual graph”
 - $h(s) = n$ and never changes
 - $h(t) = 0$
 - For all (v, w) in the residual graph $h(v) \leq h(w) + 1$
- Note maintaining the optimality condition used to show that shortest augmenting path algorithm found the maximum flow while working toward satisfying the conservation constraints



Push Re-label Algorithm

- Complexity
- $O(n^3)$ – very fast and easy to implement
 - Pushes is $O(n^3)$
 - Relabel is $O(n^2)$
 - Finding $\max \alpha_f(v)$ in constant time



Application of Network Flow

- **Edge-Disjoint Paths**
- The easiest applications of maximum flows is computing the maximum number of edge-disjoint paths between two specified vertices s and t in a directed graph G using maximum flows.
- A set of paths in G is edge-disjoint if each edge in G appears in at most one of the paths; several edge-disjoint paths may pass through the same vertex, however.
- If we give each edge capacity 1, then the maximum flow from s to t pushes either 0 or 1 units of flow along each edge. The flow-decomposition theorem implies that the subgraph S of saturated edges is the union of several edge-disjoint paths and cycles.
- Moreover, the number of paths in this decomposition is exactly equal to the value of the flow. Extracting the actual paths from S is straightforward—follow any directed path in S from s to t , remove that path from S , and recurse.



Application of Network Flow

- Edge-Disjoint Paths
- Bipartite Matching
- Tuple Selection
- Exam scheduling
- Disjoint-Path Covers
- Minimal Faculty Hiring
- Baseball Elimination
- Project selection

<https://www.youtube.com/watch?v=GhJwOIJ4SgU>

<https://www.youtube.com/watch?v=28iDs-x74OE>

<https://www.youtube.com/watch?v=HWHjQdNC-7Y>

<https://www.youtube.com/watch?v=NIQqmEXuiC8>

<https://www.youtube.com/watch?v=x2BdRml5lmc>

<https://www.youtube.com/watch?v=zrGnYstL4ss>

<https://www.youtube.com/watch?v=zrGnYstL4ss>



Maximum Matching Problem

- **Bipartite Graph**
- A **bipartite graph** (or **bigraph**) is a graph whose vertices can be divided into two disjoint and independent sets U and V , that is every edge connects a vertex in U to one in V .
- Vertex sets U and V are usually called the parts of the graph.
- Equivalently, a bipartite graph is a graph that does not contain any **odd-length cycles**

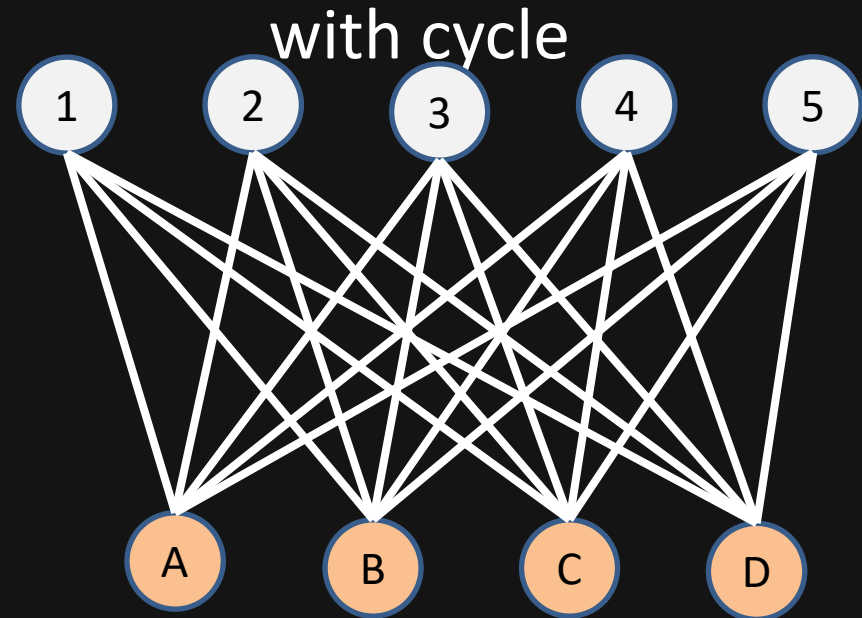
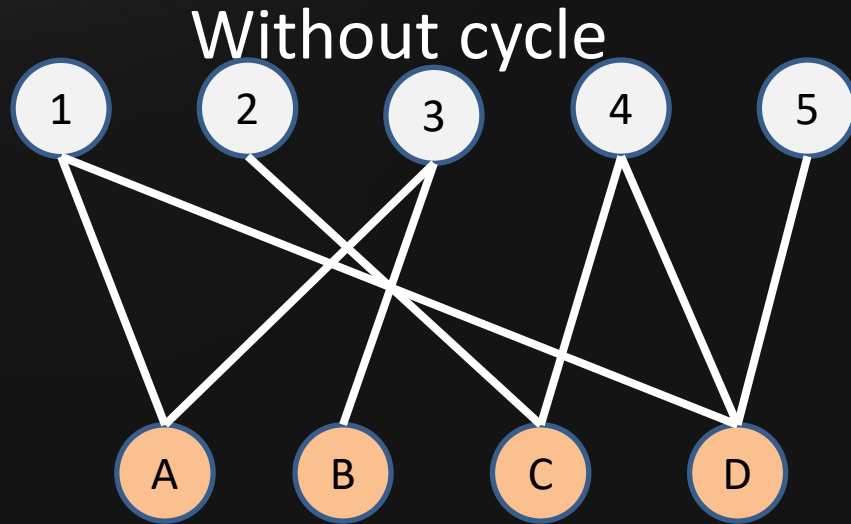


Maximum Matching Problem

- **Bipartite Graph**
- The two sets U and V may be thought of as a coloring of the graph with two colors: if one colors all nodes in U blue, and all nodes in V red, each edge has endpoints of differing colors, as is required in the graph coloring problem.
- In contrast, such a coloring is impossible in the case of a non-bipartite graph, such as a triangle: after one node is colored blue and another red, the third vertex of the triangle is connected to vertices of both colors, preventing it from being assigned either color.

Maximum Matching Problem

- Bipartite Graph



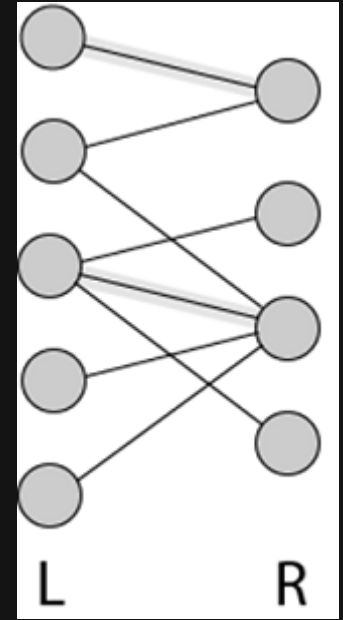


Maximum Matching Problem

- **Bipartite Graph**
- $G = (U, V, E)$ to denote a bipartite graph whose partition has the parts U and V , with E denoting the edges of the graph. If a bipartite graph is not connected, it may have more than one bipartition.
- The (U, V, E) notation is helpful in specifying one particular bipartition that may be of importance in an application. If $|U| = |V|$, that is, if the two subsets have equal **cardinality**, then G is called a **balanced bipartite graph**.
- If all vertices on the same side of the bipartition have the same degree, then G is called **biregular**.

Maximum Matching Problem

- **Maximum Bipartite Graph**
- A Bipartite Graph is a graph whose vertices can be divided into two independent sets L and R such that every edge (u, v) either connect a vertex from L to R or a vertex from R to L . In other words, for every edge (u, v) either $u \in L$ and $v \in R$. We can also say that no edge exists that connect vertices of the same set.





Maximum Matching Problem

- **Maximum Bipartite Graph**
- Matching in a Bipartite Graph is a set of edges chosen in such a way that no two edges share an endpoint. Given an undirected Graph $G = (U, V, E)$, a Matching is a subset of edge $M \subseteq E$ such that for all vertices $v \in V$, at most one edge of M is incident on v .
- A Maximum matching is a matching of maximum cardinality, that is, a matching M such that for any matching M' , we have $|M| \geq |M'|$.



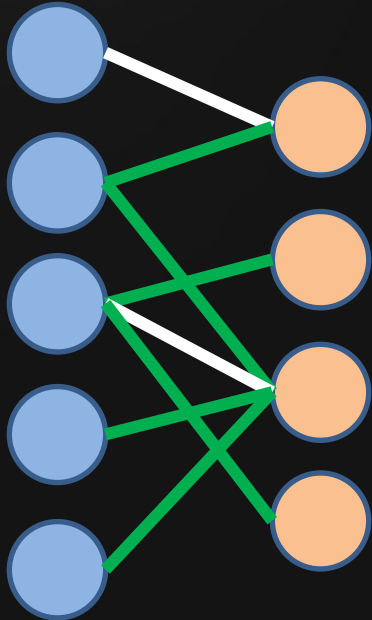
Maximum Matching Problem

- Finding a Maximum Bipartite Graph
- We can use the Ford-Fulkerson method to find a maximum matching in an undirected bipartite graph $G = (V, E)$ in time polynomial in $|V|$ and $|E|$.
- The trick is to construct a flow network $G = (V', E')$ for the bipartite graph G as follows.
- We let the source s and sink t be new vertices not in V , and we let $V' = V \cup \{s, t\}$.
- If the vertex partition of G is $V = L \cup R$, the directed edges of G' are the edges of E , directed from L to R , along with $|V|$ new directed edges:
$$E' = \{(s, u): u \in L\} \cup \{(u, v): (u, v) \in E\} \cup \{(v, t): v \in R\}$$

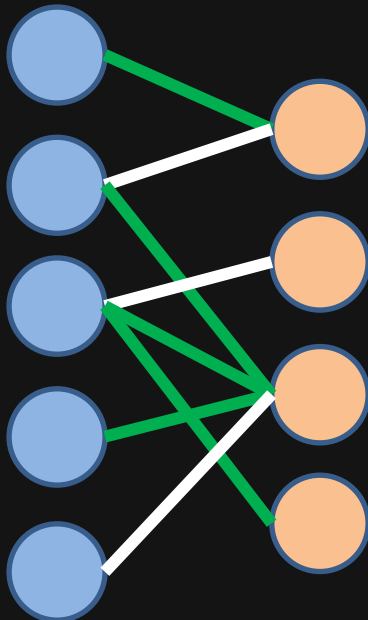
Maximum Matching Problem

- Finding a Maximum Bipartite Graph

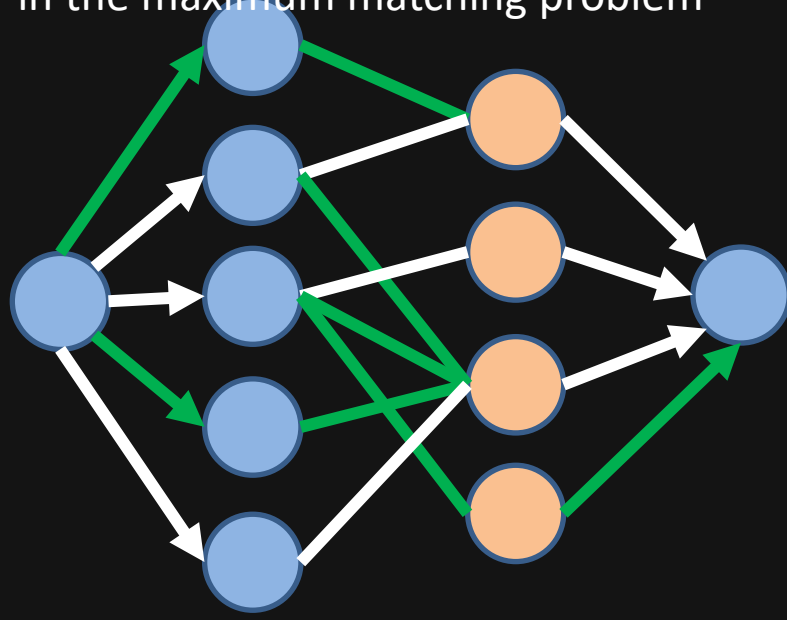
Matching Cardinality = 2



Matching Cardinality = 3



Flow Network G' with maximum flow. Each Edge has unit capacity. The white edged from L to R correspond to those in the maximum matching problem



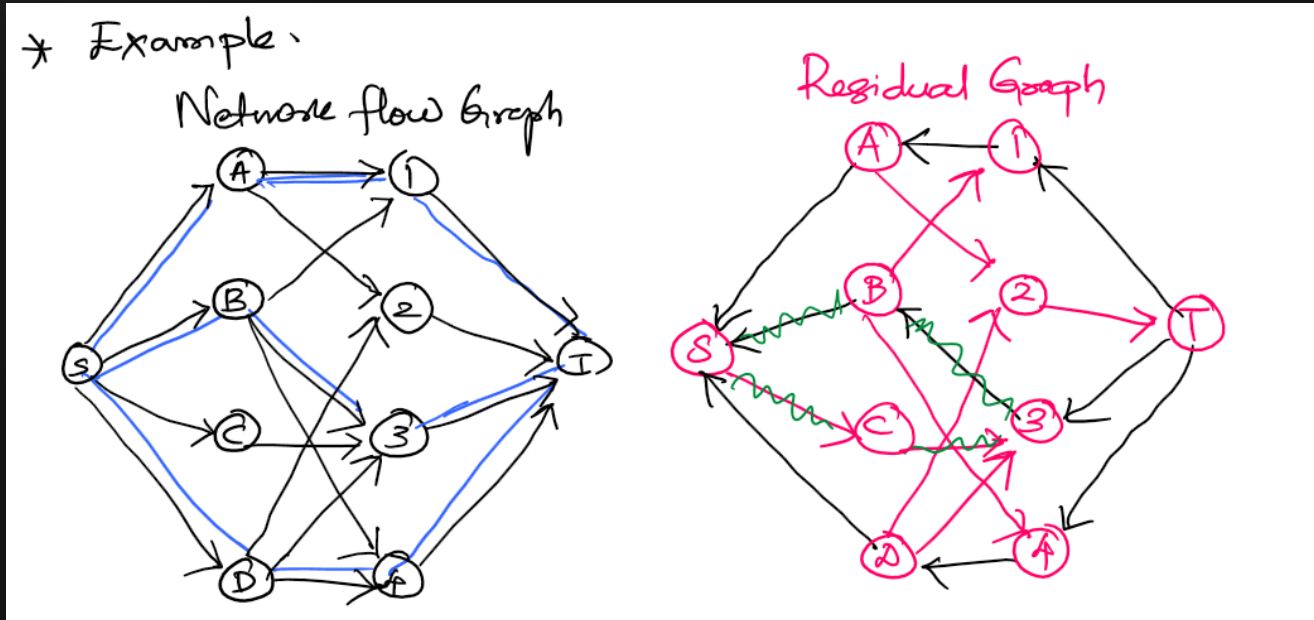


Maximum Matching Problem

- Finding a Maximum Bipartite Graph
- Bipartite Directed Graph
 - Unweighted edges
 - Max flow algorithms
 - Repeated augmenting path with DFS
 - Hopcraft-Karp
 - Weighted Edges
 - Min Cost Max flow algorithms
 - Hungarian Algorithm
 - LP network simplex

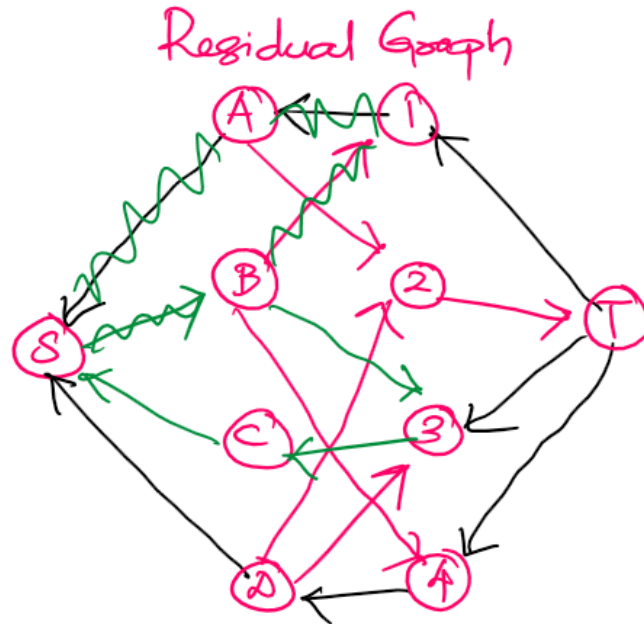
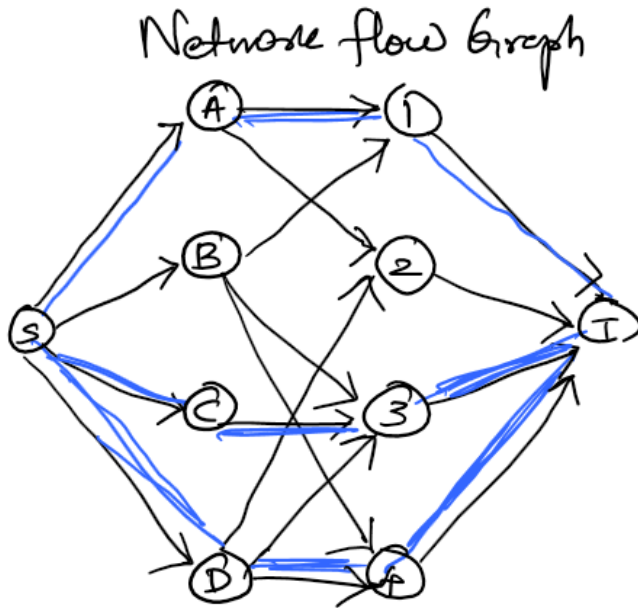
Maximum Bipartite Matching

- Unweighted Bipartite Graph – Maximum Matching



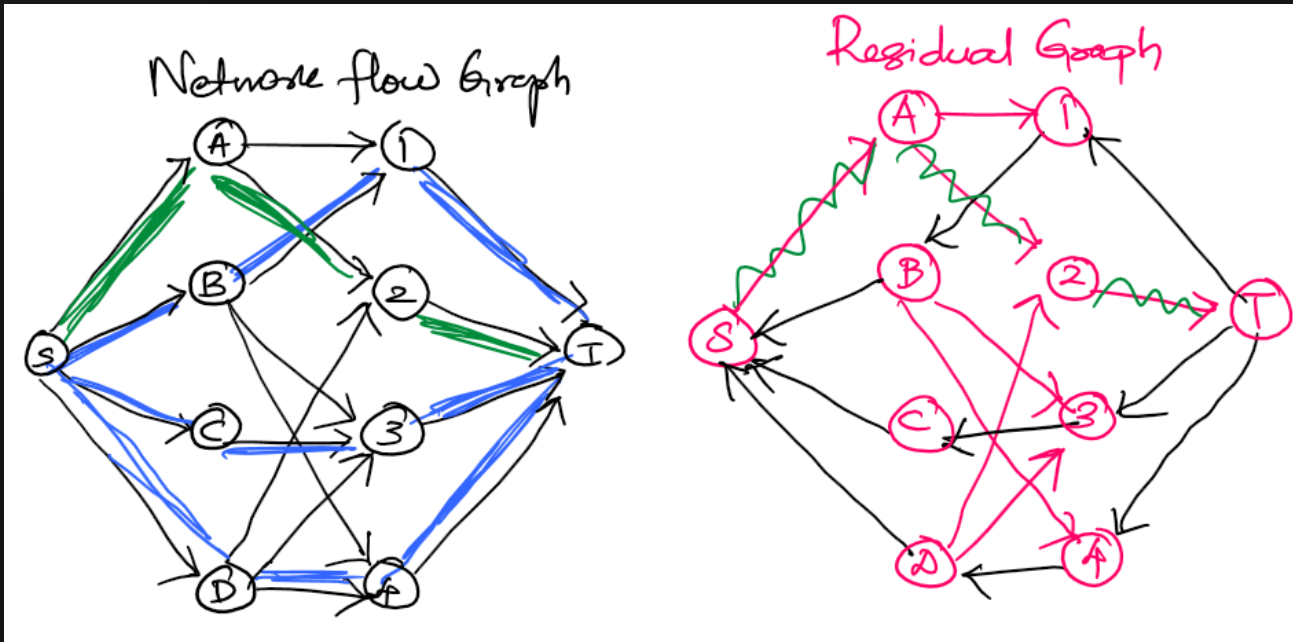
Maximum Bipartite Matching

- Unweighted Bipartite Graph – Maximum Matching



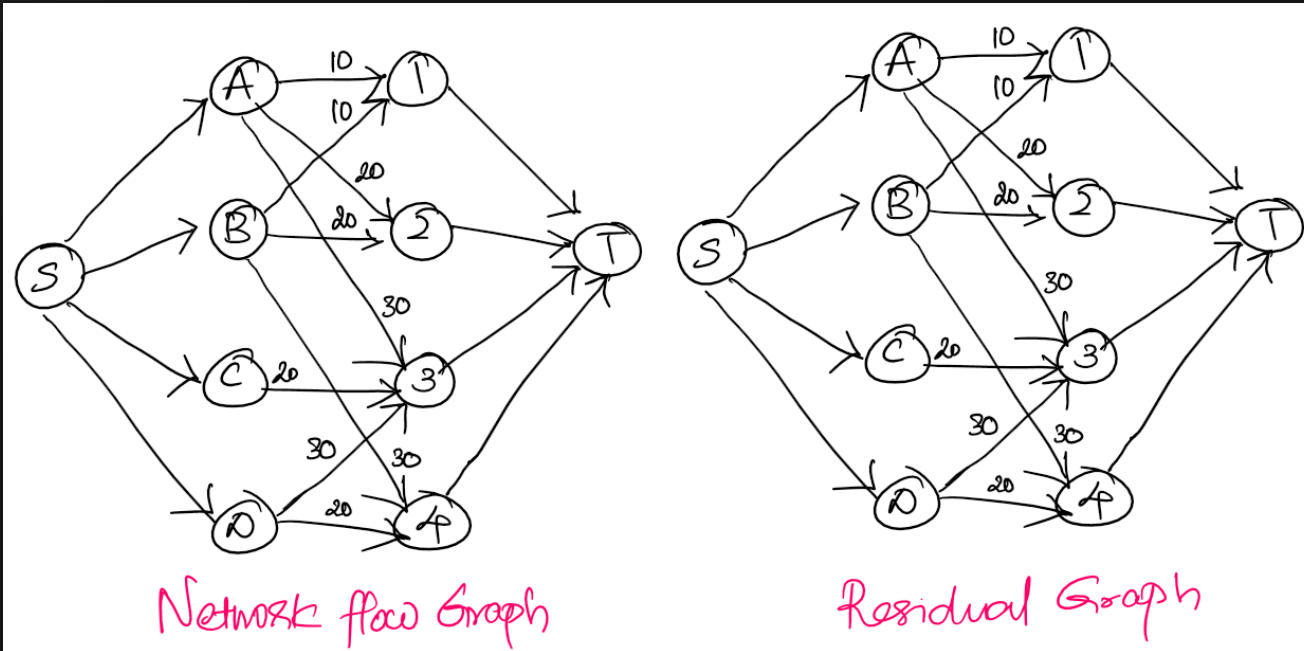
Maximum Bipartite Matching

- Unweighted Bipartite Graph – Maximum Matching



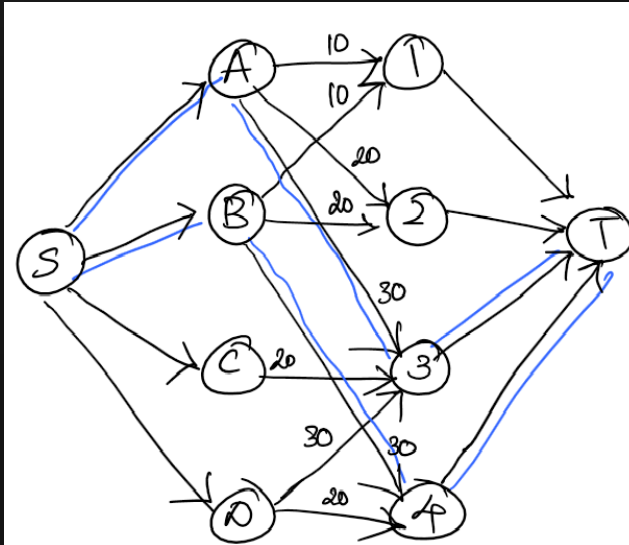
Maximum Bipartite Matching

- Weighted Bipartite Graph – Maximum Matching

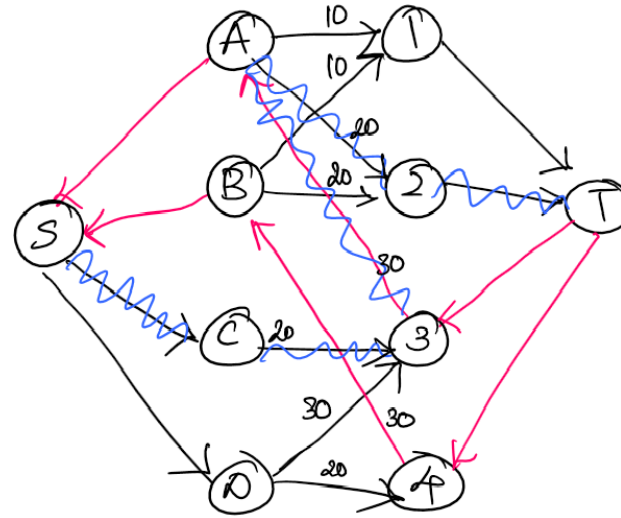


Maximum Bipartite Matching

- Weighted Bipartite Graph – Maximum Matching



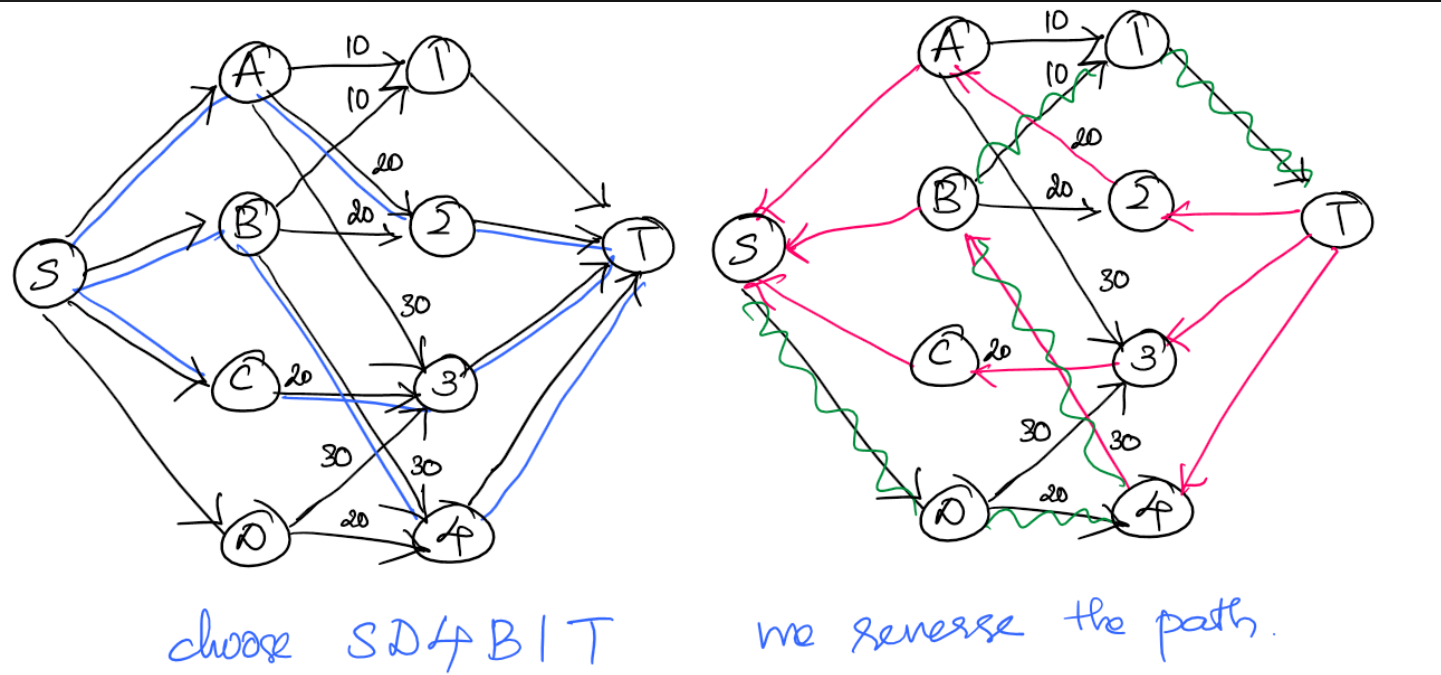
Network flow Graph



Residual Graph

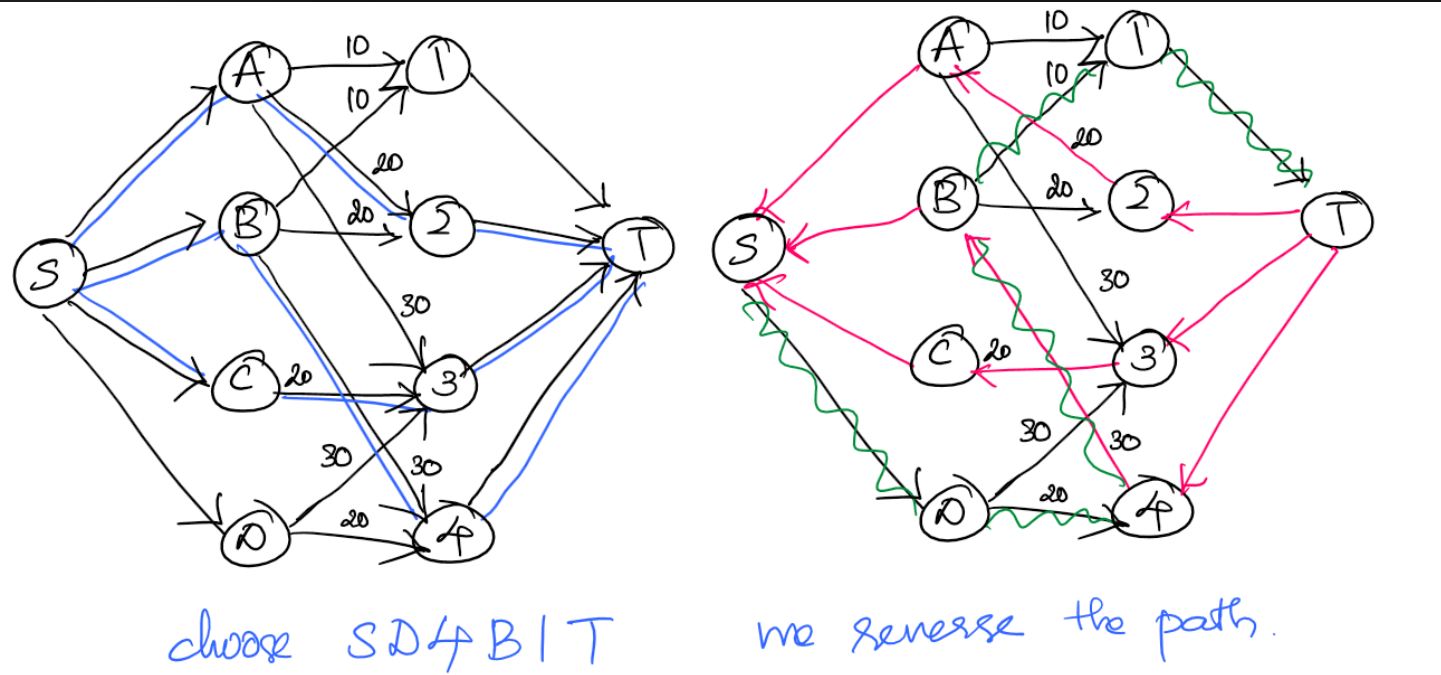
Maximum Bipartite Matching

- Weighted Bipartite Graph – Maximum Matching



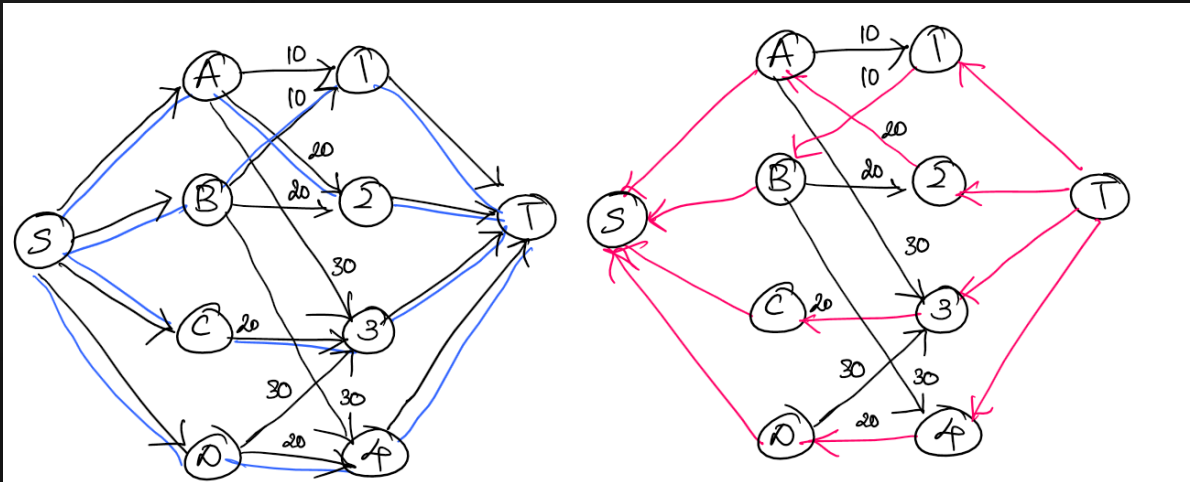
Maximum Bipartite Matching

- Weighted Bipartite Graph – Maximum Matching



Maximum Bipartite Matching

- Weighted Bipartite Graph – Maximum Matching



So Maximum matching is obtain using maximum flow
The path are $SA2T$ $SB1T$ $SC3T$ $SD4T$.



Maximum Matching Problem

- Finding a Maximum Bipartite Graph
- Given bipartite graph $G=(U,V,E)$, direct the edges from U to V
 - Add new vertices s and t .
 - Add an edge from s to every vertex in U .
 - Add an edge from every vertex in V to t .
 - Make all the capacities 1.
 - Solve maximum network flow problem on this new graph G'
 - The edge used in the maximum network flow will correspond to the largest possible matching.



Maximum Matching Problem

bipartiteMatch(u, visited, assign)

INPUT: Starting node, visited list to keep track, assign the list to assign node with another node.

OUTPUT – Returns true when a matching for vertex u is possible.

BEGIN

FOR all vertex v, which are adjacent with u, **DO**

IF v is not visited, **THEN**

 mark v as visited

IF v is not assigned, or bipartiteMatch(assign[v], visited, assign) is true, **THEN**

 assign[v] := u

RETURN true

DONE

RETURN false

END



Maximum Matching Problem

maxMatch(graph)

INPUT – The given graph.

OUTPUT – The maximum number of the match.

BEGIN

initially no vertex is assigned

count := 0

FOR all applicant u in M , **DO**

make all node as unvisited

IF bipartiteMatch(u , visited, assign), **THEN**

increase count by 1

DONE

END



Maximum Matching Problem

- **Complexity**
- In a bipartite graph $|E|$ is at most $N \cdot M$ and V will be $O(N+M)$, thus your DFS is going to take $O(NM)$. The total time complexity is then $O(NM^2)$

Module 4

Graph Algorithm

BCSE204L – Design and Analysis of Algorithm

